ELEC4848 Senior Design Project 2025-2026

Department of Electrical and Electronic Engineering

Ng Tsz Chiu (3036140210)

AI Computer Vision for Abnormal Detection in Home Pet Care Monitoring

Supervisor: Dr. W. T. Fok

Second Examiner: Dr. Andrew H.C. Wu

Date of Submission: April 20, 2026

Abstract

The rapid growth of professional in-home pet sitting has created an accountability gap: homeowners lack real-time oversight of how staff interact with their pets or respect restricted areas. This project addresses this by combining AI-powered computer vision with proximity monitoring to detect human–pet interactions and enforce spatial boundaries.

The system uses a **YOLOv8s model** fine-tuned on an expanded Roboflow dataset of 4,220 annotated images (3,736 training / 307 validation / 177 testing), with data augmentation to improve generalization. The prototype integrates an **ESP32-S3 microcontroller** with **OV5640 camera** for video streaming, **dual BLE beacons** for multi-zone proximity sensing via RSSI, a **MacBook backend** for AI inference with optimized network architecture, and an **iOS app** for live viewing, recording, and alerts. The system is implemented as a wearable chest-mount device with active thermal management to sustain continuous operation.

Evaluation achieved an overall **mAP@50 of 92.5%** and **mAP@50-95 of 77.1%**, with **precision of 94.5%** and **recall of 85.4%**. Per-class results: human detection at 94.2% mAP@50 (93.1% precision, 89.4% recall) and cat detection at 90.8% mAP@50 (95.9% precision, 81.4% recall). System testing confirmed correct triggering of automated recording upon simultaneous human+cat detection or real-time zone alerts (~1 m, RSSI-based). Key limitations include RSSI instability from physical obstructions. Future work should focus on dataset expansion, video-sequence training, and cloud deployment.

Acknowledgement

The author would like to express sincere gratitude to Dr. W. T. Fok for his supervision and guidance throughout this project, and to Dr. Andrew H.C. Wu for his role as second examiner. Thanks are also extended to the Department of Electrical and Electronic Engineering at the University of Hong Kong for providing laboratory access and resources.

Table of contents

Abstract i

Acknowledgement ii

List of Figures vi

List of Tables ix

Abbreviations x

1. Introduction 1

1.1 Background 1

1.2 Problem Definition 2

1.3 Significance 3

1.4 Scope of Work and Objectives 3

1.4.1 Primary Objectives: 3

1.4.2 Scope Boundaries: 4

1.5 Deliverables 4

1.5.1 Hardware Deliverables: 4

1.5.2 Software Deliverables: 4

1.6 Report Outline 5

2. Methodology 6

2.1 AI Model Development and Training 6

2.1.1 Dataset Selection 6

2.1.2 Training Framework and Computational Infrastructure 7

2.1.3 Model Architecture Selection 7

2.1.4 Training Hyperparameters 8

2.1.5 Data Augmentation Strategy 9

2.2 Hardware System Design 10

2.2.1 Wearable Form Factor 10

2.2.2 Thermal Management 11

2.2.3 Integrated Power Management 12

2.2.4 Multi-Zone BLE Beacon Deployment 13

2.2.5 Camera and Microcontroller Integration 16

2.3 Backend Server Architecture 17

2.3.1 Network Architecture 17

2.3.2 AI Model Integration and Inference Pipeline 18

2.4 System Integration and Data Flow 21

2.4.1 Video Frame Streaming Pipeline 21

2.4.2 RSSI-Based Distance Estimation and Zone Alerting 23

2.5 iOS Application Design 24

2.5.1 User Interface Architecture 24

3. Results and Discussion 26

3.1 AI Model Performance 26

3.1.1 Training Results Overview 26

3.1.2 Performance Progression Analysis 28

3.1.3 Per-Class Detection Accuracy 33

3.1.4 Model Testing and Real-World Validation 35

3.2 Hardware System Performance 39

3.2.1 Wearable Device Operation 39

3.2.2 Thermal Performance with Dual-Layer Cooling 45

3.2.3 Power Consumption Analysis 49

3.2.4 Multi-Zone BLE Beacon Coverage 51

3.3 System Integration and Real-Time Operation 54

3.3.1 Frame Processing and Latency 54

3.3.2 Server Stability and Endpoint Performance 58

3.3.3 ESP32-S3 Camera and Beacon Operation 61

3.4 iOS Application Performance 65

3.4.1 Live Stream Playback 65

3.4.2 Zone Alert Functionality 70

3.5 System Integration Testing 76

3.5.1 End-to-End Functionality 76

4. Conclusion 81

5. References 83

6. Appendices 85

A.1 Project Source Code Repository 85

A.2 Training Dataset 86

A.3 Training Configuration Details 87

A.4 Endpoint Specification 88

A.5 Multi-Beacon Configuration 90

A.6 System Architecture Diagram 91

List of Figures

Figure 1. Dataset created on Roboflow platform 6

Figure 2. Chest-mount harness 10

Figure 3. Final product after integrated chest-mount harness, ESP32-S3, OV5640 camera, and power bank 10

Figure 4. Cooling Fins 11

Figure 5. 40×40×10mm 5V cooling fans 11

Figure 6. ESP32-S3 added cooling fins and fans 11

Figure 7. 10,000 mAh movable power bank 12

Figure 8. BLE beacon settings via RLBeacon Tool app 13

Figure 9. BLE beacon with NRF52810 chip 14

Figure 10. Power mode and matching broadcast interval [10] 14

Figure 11. Broadcast interval and battery lifetime relationship [10] 14

Figure 12. Beacon 1 15

Figure 13. Beacon 2 15

Figure 14. ESP32-S3 module with OV5640 camera component 16

Figure 15. Settings in Arduino IDE before uploading firmware 16

Figure 16. Backend system architecture 18

Figure 17. Room mapping dictionary for beacon-to-label conversion 19

Figure 18. Example proximity alert payload structure sent to iPhone client . 20

Figure 19. NetworkManager class handling backend communication and state management 24

Figure 20. Tracking panel in terminal while training 26

Figure 21. Terminal output once finished the training 27

Figure 22. Result of Early Epochs (1–5) 28

Figure 23. Result of Mid Epochs (6–15) 29

Figure 24. Result of Later Epochs (16–20) 30

Figure 25. Result of Later Epochs (38–39) 30

Figure 26. Result of Final Epochs (40–43) 31

Figure 27. Result of Final Epochs (97-100) 31

Figure 28. Webcam-based qualitative model validation 35

Figure 29. ESP32S3+OV5640+iOSapp based qualitative model validation (human) 36

Figure 30. ESP32S3+OV5640+iOSapp based qualitative model validation (cat) 36

Figure 31. Model testing via OV5640 camera in low-light environments 37

Figure 32. Model testing when the cat is more than 1 meter away from camera 37

Figure 33. Demo of agent wearing the device 39

Figure 34. Camera view at default angle (90° horizontal) 40

Figure 35. showing the thing that to roll so that to able to change the angle .. 41

Figure 36. showing the angle that to change so that to change the angle 41

Figure 37. Device configuration at 45° camera angle 42

Figure 38. Camera view at 45° horizontal angle 42

Figure 39. Device configuration at 70° camera angle 43

Figure 40. Camera view at 70° horizontal angle 43

Figure 41. Terminal info when running the backend server 54

Figure 42. Terminal info when backend server starts to track frames posted to endpoint and collect successfully and post to /frame endpoint 55

Figure 43. Terminal frames info when powering on the ESP32-S3 61

Figure 44. Terminal distance info when powering on the ESP32-S3 62

Figure 45. “PetGuard” app 65

Figure 46. Live streams page of iOS app 65

Figure 47. Recordings page of iOS app 66

Figure 48. Pop-up alert message when the beacon is too close to the ESP32-S3 70

Figure 49. Showing the warning message and start recording 70

Figure 50. Pop-up "Room1" alert message when beacon 1 distance <1m 72

Figure 51. Pop-up "Room2" alert message when beacon 2 distance <1m 72

Figure 52. Pop-up "Room 1 and Room 2" alert message when both beacons distance <1m 73

Figure 53. iOS app showing successful detection of cat and human with recording started 76

Figure 54. BLE beacon placed on the floor in front of the room 77

Figure 55. Proximity alert pop-up displayed on the iOS app when the ESP32-S3 is within 1 meter of the beacon 77

Figure 56. iOS live stream view confirming beacon detection and active recording (distance < 1 m) 78

Figure 57. Recorded videos list in the iOS app 78

Figure 58. Playback of the selected recorded video 78

Figure 59. System Data Flow 91

List of Tables

Table 1. Selected Training Hyperparameters and Rationales 8

Table 2. Augmentation Techniques Applied During Training 9

Table 3. AI model detection accuracy results by class 33

Table 4. Thermal Performance with Dual-Layer Cooling 46

Table 5. System Power Budget (Component-Level) 49

Table 6. Beacon Placement Test Results 51

Table 7. List of endpoints 58

Table 8. Alert Mechanism Testing 71

Table 9. YOLOv8s Model Training Hyperparameters 87

Table 10. Backend Server API Endpoints 89

Table 11. BLE Beacon Hardware Configuration 90

Abbreviations

Symbol/Abbreviation	Full Name/Description
AI	Artificial Intelligence
BLE	Bluetooth Low Energy
YOLO	You Only Look Once
v8s	version 8 small
mAP	mean Average Precision
mAP50	mean Average Precision at 50% Intersection over Union
HTTP	Hypertext Transfer Protocol
iOS	iPhone Operating System
Wi-Fi	Wireless Fidelity
SSID	Service Set Identifier
UUID	Universally Unique Identifier
dBm	decibel-milliwatts
MAC	Media Access Control
RSSI	Received Signal Strength Indication
GFLOPs	Giga Floating Point Operations Per Second
C2f	Coarse-to-Fine (module)
lr0	Initial Learning Rate
lrf	Final Learning Rate
AP Isolation	Access Point Isolation

1. Introduction

1.1 Background

AI computer vision has emerged as a powerful tool for abnormal detection across various applications, from public safety to personal security. Hussain et al. [1] demonstrated deep learning approaches for cat activity classification using sensor fusion techniques. However, existing solutions focus primarily on pet behavior analysis rather than staff accountability during home visits, representing a significant gap this project aims to address.

The pet care industry continues to experience rapid growth, driven by changing social attitudes toward pet ownership. As noted in [2], "the view of pets as family members has led to a pet parenting trend where pet owners are more conscious of the need to provide specialised care" (p. 65). This perspective is reinforced by Brugger [3], who notes that as pets are increasingly perceived as family members rather than mere animals, owners have significantly increased both veterinary care frequency and overall investment in pet health. Consequently, pet owners are investing more resources and attention into comprehensive pet care solutions.

Pet owners often require temporary care for their pets due to work, travel, or emergencies. While options such as pet hotels or help from friends and family exist, they present limitations including high costs, stressful unfamiliar environments, and unavailability during urgent cases. Professional pet sitting services address these limitations by providing in-home, personalized care in familiar environments, which reduces stress for pets compared to institutional facilities. However, most pet sitting services lack monitoring systems that allow pet owners to verify caregiver conduct and ensure household boundaries are respected. This accountability gap prevents many pet owners from fully trusting professional pet sitting services, particularly in emerging markets like Hong Kong where such services are still developing.

1.2 Problem Definition

Despite advances in computer vision technology and the growing importance of professional pet care services, existing monitoring solutions do not adequately address the specific accountability needs of pet owners during home visits. This project addresses the following specific challenges:

1. **Lack of Real-Time Oversight**: Pet owners cannot verify how professional caregivers interact with their pets during home visits.
2. **Unauthorized Area Access**: There is no mechanism to detect and prevent staff from entering multiple restricted zones (e.g., bedrooms, storage areas).
3. **Behavioral Accountability**: Without video evidence, disputes about pet care or safety incidents cannot be resolved objectively.
4. **System Integration Complexity**: Existing monitoring solutions lack integrated hardware-software systems suitable for wearable deployment in dynamic home environments.

These challenges represent a significant gap in pet care accountability, particularly in Hong Kong where mature pet monitoring solutions remain limited. This project aims to bridge this gap by developing an integrated hardware-software platform that combines real-time video monitoring with multi-zone proximity detection.

1.3 Significance

This project contributes to pet care accountability and owner peace-of-mind by:

- Protecting Animal Welfare: Automated detection of human-pet interactions enables early identification of improper handling.
- Enhancing Security: Real-time alerts for restricted zone breaches protect both pets and client property, with support for multiple restricted areas.
- Building Trust: Transparent video records and alerts strengthen the relationship between pet owners and service providers.
- Establishing a Model: The integrated hardware-software platform demonstrates feasibility of AI-powered personal security wearables for emerging markets.

1.4 Scope of Work and Objectives

1.4.1 Primary Objectives:

1. Develop an AI computer vision model capable of detecting humans and cats with high accuracy in real-time.
2. Implement a wearable hardware system integrating camera, microcontroller, proximity sensors, and thermal management.
3. Create a backend server that processes video frames and manages data flow between hardware and user interface with optimized latency.
4. Design an iOS application enabling remote monitoring, video playback, and room-specific zone-based alerts.
5. Integrate multi-beacon BLE-based proximity sensing for enforcing multiple restricted zones within a single home.

1.4.2 Scope Boundaries:

- System designed for indoor residential environments.
- Testing conducted on Wi-Fi networks without AP Isolation restrictions.
- Focus on binary detection (human/cat presence) rather than behavior classification.
- Support for up to 2 restricted zones via dual BLE beacon configuration.

1.5 Deliverables

1.5.1 Hardware Deliverables:

- Wearable chest-mount device containing ESP32-S3, OV5640 camera, power management system, active cooling fans, and passive cooling fins
- Dual BLE beacons (Beacon 1 for Room 1, Beacon 2 for Room 2) configured for multi-zone coverage testing
- Integrated thermal management system with both active cooling fans and passive cooling fins for extended operation
- Movable power bank charger providing unified power supply to ESP32-S3 and cooling fans

1.5.2 Software Deliverables:

- Trained YOLOv8s model (best.pt) with 4,220-image dataset (3,736 training, 307 validation, 177 test).
- Optimized backend server with WebSocket-based network architecture which contains Trained YOLOv8s model and endpoints for data transaction and detect processing.
- Multi-beacon proximity detection with room-specific alert messages.
- iOS application (PetGuard) with live streaming, recording management, and room-aware zone alert features.

1.6 Report Outline

This report is structured as follows:

- Section 1 (Introduction): Establishes project context, problem definition, and objectives.
- Section 2 (Methodology): Details the choices, methods, and rationale for dataset, training frameworks, hardware configuration, thermal management, network optimization, and system design.
- Section 3 (Results and Discussion): Presents experimental outcomes with measured performance metrics and analytical commentary on findings and limitations.
- Section 4 (Conclusion): Summarizes achievements, contributions, limitations, and recommendations for future work.
- Appendices: Source code repository and additional technical resources.

2. Methodology

This project implements a real-time pet-sitting monitoring system that combines computer vision-based human–cat detection with BLE beacon-based proximity alerting. The system comprises four integrated components: (1) a YOLOv8s detection model trained on 4,220 annotated images (3,736 training / 307 validation / 177 testing) using transfer learning and data augmentation, (2) a chest-mounted wearable platform integrating an ESP32-S3 microcontroller, OV5640 camera module, dual-layer thermal management, and a centralized 10,000 mAh power supply, (3) a Python backend server that receives camera frames and beacon distance data, performs real-time inference, and streams annotated results to clients via a low-latency networking pipeline (HTTP ingestion with WebSocket delivery), and (4) an iOS application developed with SwiftUI that provides live monitoring, recording playback, and multi-zone proximity alerts. The following sections describe the design, implementation, and integration of each component.

2.1 AI Model Development and Training

2.1.1 Dataset Selection

A custom dataset of 4,220 annotated images was assembled using the Roboflow platform (Figure 1) and split into training (3,736), validation (307), and testing (177) sets, with two classes: "human" and "cat." Images were collected from diverse cat breeds and various human body parts to ensure broad coverage. Roboflow was selected for its user-friendly annotation tools, native YOLOv8 pipeline support, and automated augmentation capabilities. The large dataset promotes diversity and supports balanced class representation to reduce training bias.

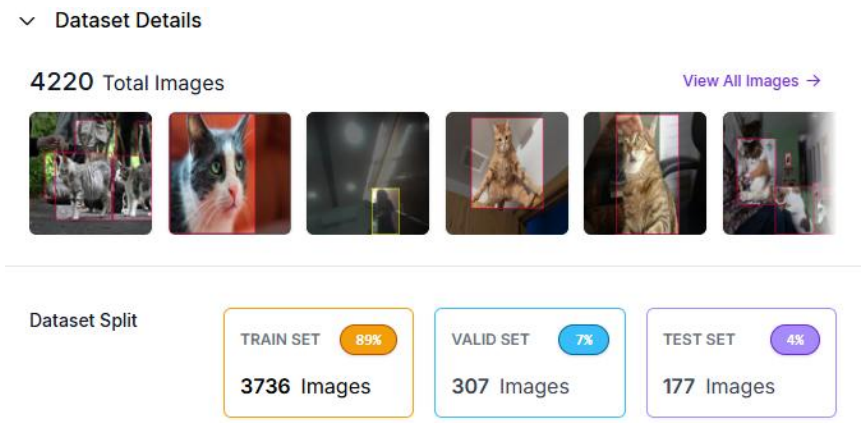

Figure 1. Dataset created on Roboflow platform

2.1.2 Training Framework and Computational Infrastructure

The model was trained using the Ultralytics YOLOv8 framework on an NVIDIA RTX 4090 GPU at the HKU Innovation Wing computer lab. With 24 GB VRAM and 16,384 CUDA cores, the RTX 4090 enabled efficient batch processing (16 images/batch) and parallel computation across 100 epochs [4]. The Ultralytics framework was chosen for its native CUDA support and seamless NVIDIA hardware integration [5].

2.1.3 Model Architecture Selection

The pre-trained YOLOv8s model was selected as the detection backbone. YOLOv8s is a single-stage detector using C2f modules, designed to balance inference speed and accuracy for real-time applications [6]. The "small" variant was chosen over larger models to meet real-time processing requirements on the MacBook backend server while maintaining low inference latency for continuous 10 FPS frame processing. Transfer learning from pre-trained COCO dataset weights [7] accelerated convergence, and at 28.6 GFLOPs the model is suitable for embedded hardware deployment.

2.1.4 Training Hyperparameters

The following hyperparameters (Table 1) were selected to optimize model training on the 4,220-image dataset:

Parameter	Value	Rationale
Epochs	100	Sufficient duration for dataset convergence
Batch Size	16	Balances gradient stability and GPU memory utilization on RTX 4090
Image Size	640px	Standard YOLO input; balances detail and compute load
Optimizer	AdamW	Adaptive learning with decoupled weight decay (lr=0.001667, momentum=0.9)
Initial Learning Rate (lr0)	0.01	Auto-adjusted to 0.001667 by optimizer=auto
Final Learning Rate (lrf)	0.01	Ratio of final to initial LR for decay schedule
Warm-up Epochs	3	Stabilizes model before full-rate training
Learning Rate Decay	Linear (epoch 3–100)	Smooth, predictable reduction of learning rate magnitude
Device	GPU (NVIDIA RTX 4090)	Enables efficient parallel training
Early Stopping Patience	20 epochs	Prevents overfitting; stops training if no improvement for 20 consecutive epochs (training completed all 100 epochs as improvement continued)

Table 1. Selected Training Hyperparameters and Rationales

A batch size of 16 provides stable gradient averaging within GPU memory constraints. The 3-epoch warm-up prevents early training instability [8], enabling smooth progression from broad exploration to fine-tuning.

2.1.5 Data Augmentation Strategy

A default Ultralytics YOLOv8 data augmentation strategy (Table 2) was applied during training to increase dataset diversity and improve model robustness:

Augmentation Type	**Configuration**	**Purpose**
Geometric Transformations	- Horizontal flip (50%, fliplr=0.5) - Vertical flip (10%, flipud=0.1) - Rotation (±15°, degrees=15) - Translation (±10% image size, translate=0.1) - Scaling (±20% range, scale=0.2) - Shear (±10°, shear=10)	Robustness to varied orientations and body positions
Color Space Transforms	- HSV modifications (hue ±5%, hsv_h=0.05; saturation ±20%, hsv_s=0.2; value ±20%, hsv_v=0.2)	Handles varying household lighting conditions
Mosaic Composition	4-image mosaicking at 0.5 probability (mosaic=0.5, disabled for last 10 epochs via close_mosaic=10)	Increases mini-batch diversity and contextual information
Random Erasing	Random erasing at 0.1 probability (erasing=0.1)	Improves robustness to partial occlusion [9]

Table 2. Augmentation Techniques Applied During Training

The geometric transforms simulate varied pet postures and human positions. Color space modifications address residential lighting variations. Mosaic composition increases training diversity, and random erasing improves robustness to partially occluded subjects.

2.2 Hardware System Design

2.2.1 Wearable Form Factor

The hardware system was designed into a chest-mounted wearable configuration to better simulate real-world deployment scenarios for pet-sitting staff. The device is composed of a custom chest-mount harness (Figure 2) that provides stable positioning while allowing natural movement. Key components integrated into the wearable include the ESP32-S3 microcontroller, OV5640 camera module, thermal management system, and a 10,000 mAh power bank (Figure 3) that serves as the unified power supply. This wearable form factor enables staff to move freely across different areas of a home during pet-sitting sessions, providing realistic behavior simulation and improved monitoring coverage.

Figure 2. Chest-mount harness

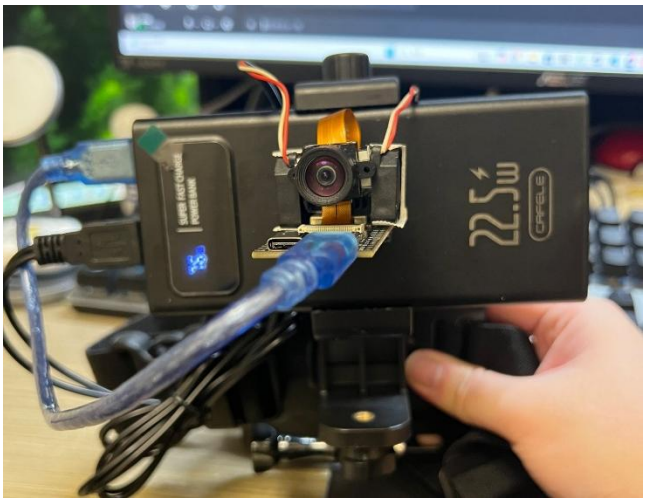

Figure 3. Final product after integrated chest-mount harness, ESP32-S3, OV5640 camera, and power bank

2.2.2 Thermal Management

To address thermal challenges during extended operation, a dual-layer thermal management system combining passive and active cooling was implemented.

Passive Cooling (Cooling Fins): Aluminum cooling fins (Figure 4) are stacked directly on the ESP32-S3 and OV5640 using thermally conductive adhesive pads, increasing surface area for passive heat dissipation with zero power consumption.

Figure 4. Cooling Fins

Active Cooling (Fans): Two 40×40×10 mm, 5V fans (Figure 5) are positioned near main heat sources, each drawing ~50 mA (100 mA total)

Figure 5. 40×40×10mm 5V cooling fans

The dual-layer system (Figure 6) ensures thermal stability: passive fins provide baseline dissipation, while active fans prevent thermal throttling during sustained operation.

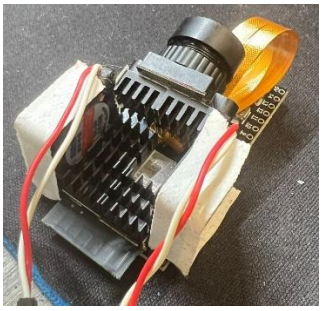

Figure 6. ESP32-S3 added cooling fins and fans

2.2.3 Integrated Power Management

A unified 10,000 mAh movable power bank (Figure 7) serves as the centralized power supply for the entire wearable system. Power is distributed to the following components:

- ESP32-S3: ~120 mA @ 5V
- OV5640 camera module: ~80 mA @ 5V
- Cooling fans (x2): ~100 mA @ 5V

Figure 7. 10,000 mAh movable power bank

Total system draw is approximately 305 mA. The movable design allows battery swapping between sessions and eliminates multiple separate battery modules.

2.2.4 Multi-Zone BLE Beacon Deployment

The system employs a dual-beacon configuration to enable multi-zone restricted area detection. Once BLE beacons are powered on, they continuously broadcast Bluetooth signals. The ESP32-S3 receives the beacon's RSSI (Received Signal Strength Indicator) and transmits it to the backend server to estimate proximity distance.

BLE beacons are configured via the RLBeacon Tool app (Figure 8) with the following specifications:

- Chip: NRF52810 (figure 9)
- Broadcast Interval: 960 ms (every 0.96 seconds)
- TX Power: Mode 4 (As shown in Figure 10, -8dBm is used which provides a suitable indoor broadcast range at low power consumption)
- Expected Battery Life: 18 months at 960 ms interval (see Figure 11)

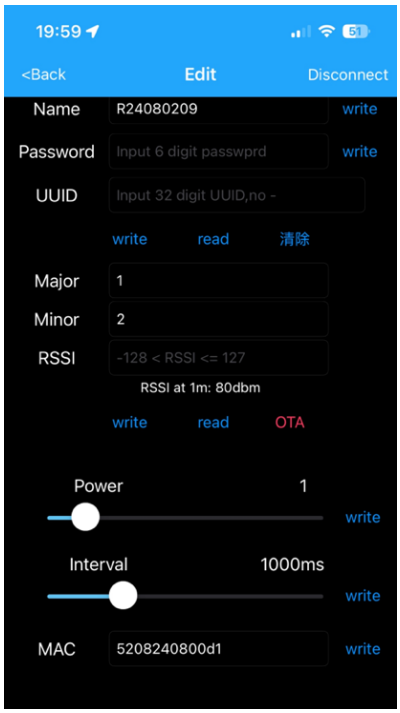

Figure 8. BLE beacon settings via RLBeacon Tool app

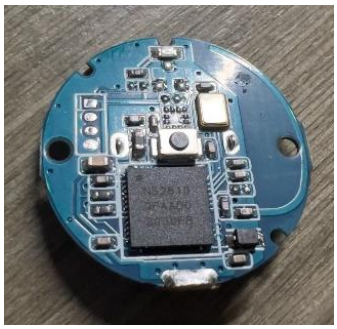

Figure 9. BLE beacon with NRF52810 chip

Power	nRF52810
1	-20dBm
2	-16dBm
3	-12dBm
4	-8dBm
5	-4dBm
6	0dBm
7	3dBm
8	4dBm

Figure 10. Power mode and matching broadcast interval [10]

3.3Battery Lifespan

Taking B2 non-sensor as an example, power 0dBm, modify broadcast interval, CR2032 battery usage time.

Broadcust Power	Coverage Range	Broadcust Interval	Battery Usage Time (Months)
0dBm	30~50m	40ms	1
		120ms	2.7
		520ms	10.2
		1000ms	18.4
		2000ms	32
		4000ms	50.2

Figure 11. Broadcast interval and battery lifetime relationship [10]

Beacon Details:

- Beacon 1 (figure 12): MAC Address 52:08:24:08:00:d1, UUID 2cac9dcafff341c782199af018c2de16
- Beacon 2 (figure 13): MAC Address 52:08:24:08:00:f9, UUID fda50693a4e24fb1afcfc6eb07647825

Figure 12. Beacon 1

Figure 13. Beacon 2

Distance Threshold Rule:

Distance estimation is based on RSSI values received by the ESP32-S3. A proximity threshold of ≤ 1.0 meter is used to detect when the wearable device enters a restricted zone. Video recording is triggered when either simultaneous human + cat detection occurs or when the device proximity to any beacon falls below the threshold.

Beacon Matching Logic (in ESP32 firmware):

1. Primary Match: Compare beacon MAC address against pre-defined target MAC addresses.
2. Fallback Match: If MAC matching is unreliable due to environmental factors, parse manufacturer data and match by UUID + Major + Minor.

2.2.5 Camera and Microcontroller Integration

The OV5640 5MP camera module is paired with the ESP32-S3 (Figure 14), connected via DVP (Digital Video Port) for raw pixel transfer and SCCB (Serial Camera Control Bus) for sensor configuration [11]. The OV5640 supports up to 1920×1080 at 30 fps. The ESP32-S3 features a dual-core Xtensa LX7 CPU at up to 240 MHz, 512 KB SRAM, and integrated 2.4 GHz Wi-Fi and Bluetooth 5 LE [12].

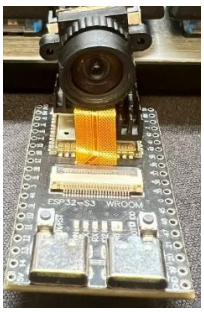

Figure 14. ESP32-S3 module with OV5640 camera component

Firmware Development: The ESP32-S3 firmware is developed and uploaded using the Arduino IDE (Figure 15). The OV5640 is natively supported by Espressif's ESP-IDF libraries, enabling efficient driver integration with minimal custom development overhead.

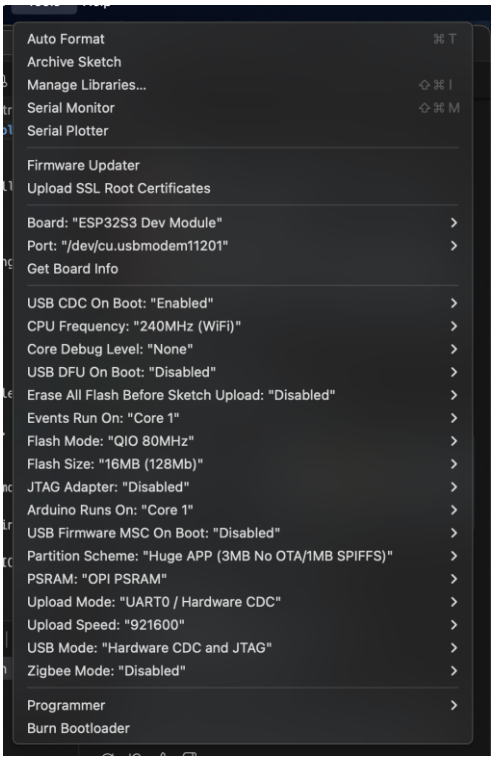

Figure 15. Settings in Arduino IDE before uploading firmware

2.3 Backend Server Architecture

2.3.1 Network Architecture

The backend implements a WebSocket-based streaming architecture with binary frame transmission. Frames from the ESP32-S3 are sent via HTTP POST to the MacBook backend for AI processing, then streamed to the iPhone client over a persistent WebSocket connection.

Key Architecture Components:

1. WebSocket Protocol: A persistent bidirectional connection enables full-duplex streaming with low protocol latency. This allows continuous frame transmission without connection overhead.

2. Binary Frame Transmission: Frames are transmitted using a compact 4-byte little-endian size header followed by raw JPEG data. This compact format minimizes bandwidth usage and transmission time.

3. Adaptive Frame Rate Control: The system continuously monitors server queue depth and dynamically adjusts the frame rate between 10-25 FPS. When queue depth is high, the frame rate is reduced to prevent backlog; when network conditions allow, it is increased.

4. Frame Skipping and Queue Management: When queue depth exceeds 50% capacity, the oldest frame is automatically skipped. This ensures the buffer maintains the most recent frames and prevents excessive buffering.

5. Asynchronous I/O and Frame Buffering: Flask endpoints use thread-based request handling (`threaded=True`) with asyncio used for the video writer coroutine. A circular buffer that stores only the latest processed frame prevents redundant processing and allows efficient delivery of the most up-to-date annotated frame.

2.3.2 AI Model Integration and Inference Pipeline

The backend server integrates the YOLOv8s AI model to handle real-time object detection and multi-beacon proximity state tracking. Figure 16 shows the overall backend system architecture.

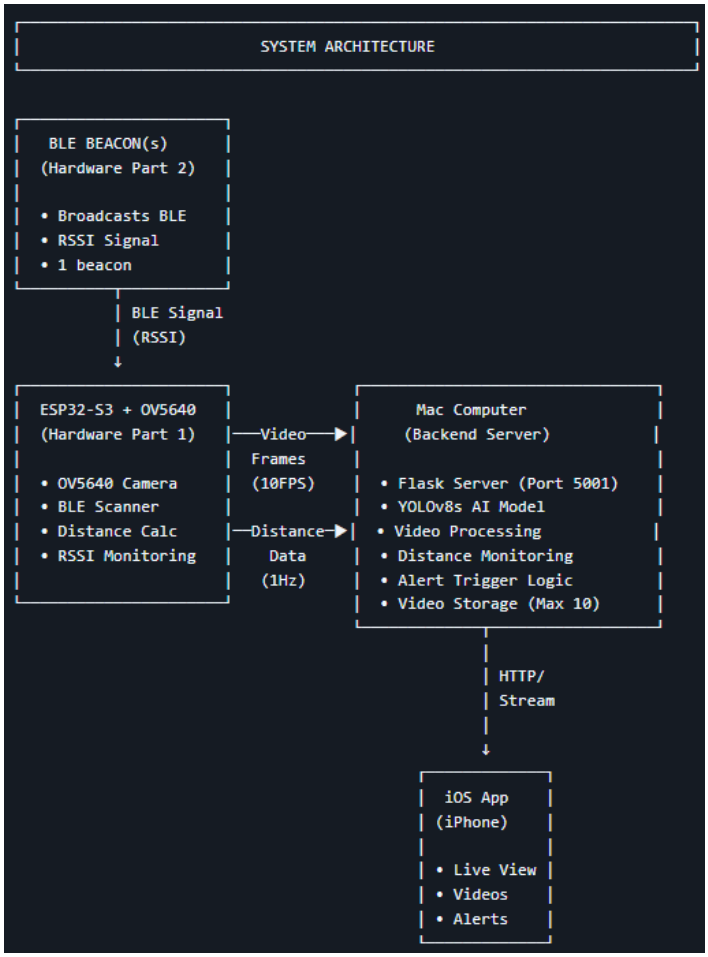

Figure 16. Backend system architecture

Inference Pipeline:

The inference pipeline processes frames and proximity data as follows:

1. Receive JPEG frames from the ESP32-S3 via the POST /esp32/frame endpoint at 10 FPS.
2. Decode the JPEG image into a NumPy array using OpenCV.
3. Perform inference using the YOLOv8s model with model.predict(frame).

4. Parse the detection outputs, including class labels, confidence scores, and bounding boxes.
5. Overlay bounding boxes and class labels on the original frame.
6. Encode the annotated frame back to JPEG format (quality: 75).
7. Store the annotated frame in a circular buffer for WebSocket streaming.
8. Detect simultaneous "human" and "cat" presence to trigger the recording flag.
9. Process multi-beacon proximity data received from the /esp32/distance endpoint, map each beacon to its corresponding room label, and identify any beacons within the 1.0-meter danger zone.
10. Generate room-specific alert messages and stream the annotated frame along with proximity status via WebSocket to connected iPhone clients.
11. Maintain legacy endpoints (/stream/live and /videos) for backward compatibility.

Multi-Beacon Proximity State Tracking:

The system maintains a proximity state including beacon_id, danger_beacon_ids, danger_beacon_labels, and proximity_alert. A room mapping dictionary converts beacon identifiers into human-readable room labels (Figure 17).

```
BEACON_LABELS = {
  'beacon_1': 'Room 1',
  'beacon_2': 'Room 2'
}
```

Figure 17. Room mapping dictionary for beacon-to-label conversion

When any beacon enters the proximity threshold (< 1.0 m), the proximity_alert flag is activated and the corresponding room label(s) are included in the alert payload. An example payload sent to the iPhone client is shown in Figure 18.

```
{
  "proximity_alert": true,
  "beacon_id": "beacon_1",
  "danger_beacon_ids": ["beacon_1"],
  "danger_beacon_labels": ["Room 1"],
  "recording": true
}
```

Figure 18. Example proximity alert payload structure sent to iPhone client

Recording Trigger Logic:

The system uses two independent recording triggers (OR logic):

- Trigger 1 (Detection): When both "human" AND "cat" are simultaneously detected in a frame, recording starts automatically regardless of proximity status.
- Trigger 2 (Proximity): When any beacon distance falls below 1.0 meter, recording starts automatically regardless of detection status.
- Recording stops when the active trigger condition is no longer met (with a 2-second cooldown for detection-based triggers).

Video Storage Management:

The system maintains a maximum of 10 recorded video files (MAX_VIDEOS = 10). When this limit is exceeded, the oldest videos are automatically deleted to free storage space. Recorded videos are saved in MP4 format at 30 FPS with filenames containing the recording timestamp.

Framework Selection:

The Ultralytics YOLOv8 library was selected for its high-level API, seamless Flask integration, and flexible model export capabilities, abstracting ONNX/TensorRT handling for rapid prototyping [5].

2.4 System Integration and Data Flow

2.4.1 Video Frame Streaming Pipeline

The system employs a hybrid streaming pipeline consisting of JPEG-over-HTTP transmission from the ESP32-S3 to the backend, followed by WebSocket-based streaming from the backend to the iPhone client, with adaptive frame rate control to optimize performance under varying network conditions.

The video frame streaming pipeline operates in three main stages:

Stage 1: Frame Capture and Compression (ESP32-S3)

1. The OV5640 camera captures frames internally at 30 fps.
2. Frames are JPEG-compressed on the ESP32-S3, reducing the data size from approximately 2.7 MB (uncompressed) to 80–100 KB per frame.
3. Compressed frames are transmitted to the backend via HTTP POST to the /esp32/frame endpoint at a base rate of 10 fps.

Stage 2: Backend Processing (MacBook)

1. The backend receives the JPEG frame from the ESP32-S3.
2. The YOLOv8s model performs inference to detect humans and cats.
3. Bounding boxes and class labels are overlaid on the frame.
4. The annotated frame is re-encoded to JPEG format at quality 75.
5. The processed frame is stored in a circular buffer, retaining only the latest frame.
6. The annotated frame is converted to binary format (4-byte size header + JPEG data) and broadcast to all connected iPhone clients via WebSocket.

Stage 3: iPhone Display (iOS App)

1. The iPhone client receives binary frames through the WebSocket connection.
2. The 4-byte size header is parsed to determine the JPEG data length.

3. The JPEG data is decoded into a UIImage.
4. The frame is rendered on the live stream view at the received frame rate.

Adaptive Frame Rate Control:

If queue depth exceeds 75%, frame rate is reduced to a minimum of 10 FPS. If queue depth falls below 25%, frame rate gradually increases to a maximum of 25 FPS.

2.4.2 RSSI-Based Distance Estimation and Zone Alerting

The system adopts an RSSI-based distance estimation and zone alerting mechanism by converting Bluetooth RSSI signal strength into distance estimates using a calibrated log-distance path loss model. The model calculates distance for each beacon using the following formula [13]:

$$\text{Distance} = 10^{\frac{\text{TX Power} - \text{RSSI}}{10 \times n}}$$

Where:

- TX Power: Nominal TX power at 1 meter (-8 dBm for NRF52810 in mode 4)
- RSSI: Measured received signal strength (detected by the esp32s3 via bluetooth)
- n: Path loss exponent (2.5 for indoor residential)

Proximity Detection and Alerting:

A distance threshold of ≤ 1.0 meter triggers the zone alerting mechanism:

- Proximity Threshold: If distance ≤ 1.0 meter for ANY beacon → proximity_alert = true
- Room-Specific Alert: Display room label from beacon (e.g., "Someone is too close to Room 1!")
- Multiple Zones: If multiple beacons in danger zone, alert lists all rooms (e.g., "Someone is too close to Room 1 and Room 2!")
- Recording Trigger: Uses OR logic — see “section 2.3.2 Recording Trigger Logic” for full details

2.5 iOS Application Design

2.5.1 User Interface Architecture

The iOS application is developed using SwiftUI in Xcode and deployed to an iPhone. It uses a tab-based navigation structure with two primary views: Live Stream and Recordings. A Settings modal is accessible from the Live Stream tab.

A central NetworkManager class (Figure 19), implemented as an ObservableObject, manages all backend communication via REST APIs and URLSession. @Published properties reactively update the UI when network data changes.

```swift
class NetworkManager: ObservableObject {
    @Published var isConnected = false
    @Published var isRecording = false
    @Published var currentVideo: String?
    @Published var detections: [Detection] = []
    @Published var proximityAlert = false
    @Published var beaconDistance: Double = 999.0
    @Published var beaconId: String?
    @Published var dangerBeaconIds: [String]?
    @Published var dangerBeaconLabels: [String] = []

    private(set) var baseURL = "http://172.20.10.3:5001"

    func updateServerURL(_ url: String) {
        self.baseURL = url
    }

    func fetchLiveFrame(completion: @escaping (UIImage?) -> Void) {
        // Fetches base64-encoded JPEG frame from /stream/live endpoint
        // Decodes to UIImage and updates @Published properties
    }
}
```

Figure 19. NetworkManager class handling backend communication and state management

Live Stream Tab:

StreamView displays real-time video frames fetched from /stream/live. Frames are received as base64-encoded JPEG, decoded into UIImage, and rendered via SwiftUI's Image component. A polling timer (0.1 s interval, ~10 FPS) fetches frames continuously. The overlay bar displays:

The Live Stream tab includes a top overlay bar displaying:

- Connection Status: Green/red indicator showing connected/disconnected state

- Proximity Alert Banner: Orange banner displaying room-specific alert messages (e.g., "ALERT: Room 1, Room 2") when dangerBeaconLabels are populated
- Recording Indicator: Red recording badge when isRecording is true
- Settings Button: Gear icon button that presents the Settings modal sheet

Recordings Tab:

The VideosView retrieves a list of recorded videos from the backend /videos endpoint and displays them in a scrollable list. Videos are sortable by date/time. Users can select and play any recorded video using the built-in AVPlayer component with variable speed playback controls (×0.5, ×1, ×2). The recordings view also supports swipe-to-delete functionality, which operates as a backend storage management feature (the backend maintains a maximum of 10 videos via the MAX_VIDEOS auto-cleanup mechanism).

Settings Modal:

The SettingsView appears as a modal sheet from the Live Stream tab using the .sheet(isPresented:) modifier. Users can configure the backend server IP and port via TextFields. A "Test Connection" button calls checkHealth() to verify connectivity, with status shown as Connected (green) or Disconnected (red). The modal also includes instructions on finding the device IP and the correct URL format (e.g., http://YOUR_DEVICE_IP:5001).

Multi-Zone Alert Handling:

The app provides multi-zone proximity alerts by tracking dangerBeaconLabels from the backend. When any beacon enters the proximity threshold, an alert displays "Someone too close to the area!" or a specific message like "Someone too close to [Room 1, Room 2]!" based on the triggered room labels.

Notification Handling:

Notifications are configured via UNUserNotificationCenter in AppDelegate. On launch, the app requests alert/sound/badge permissions. When proximity_alert = true, both visual on-screen alerts and push notifications are triggered.

3. Results and Discussion

The results of this project are satisfactory. The AI model, wearable hardware, backend server, and iOS application were successfully integrated and operated as intended, enabling real-time monitoring, multi-zone proximity alerting, and automatic video recording. This section reports observed performance for each subsystem, discusses implementation difficulties, and outlines current limitations and potential improvements.

3.1 AI Model Performance

3.1.1 Training Results Overview

Training Configuration Summary:

- Settings Button: Gear icon button that presents the Settings modal sheet
- Dataset: 4,220 images (3,736 training / 307 validation / 177 test)
- Hardware: NVIDIA RTX 4090 at HKU computer lab
- Total Training Time: 1.659 hours for 100 epochs (~60 s/epoch)
- Training Speed: ~5.6 iterations/second
- Final Model: weights/best.pt (best validation performance)
- Parameters: 11,136,374 trainable (28.6 GFLOPs)
- Validation Instances: 420 across 307 images

During the training of YOLOv8s using the script "train_model.py" written for this project, the terminal kept track of the training progress and displayed the result of each epoch (Figure 20). Trained YOLOv8s model achieving 92.5% mAP@50 and 85.4% recall on validation set, with per-class performance of 90.8% mAP@50 for cats and 94.2% mAP@50 for humans (Figure 21).

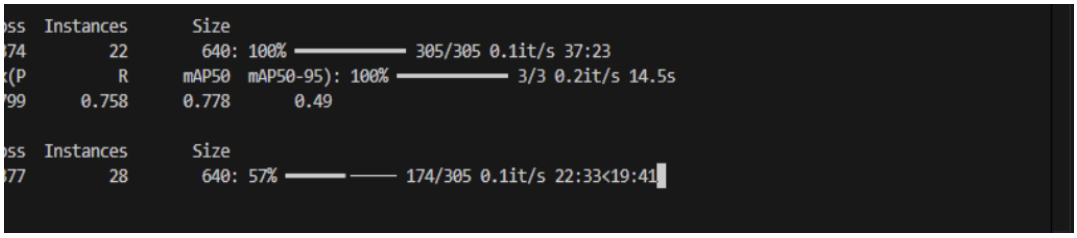

Figure 20. Tracking panel in terminal while training

```
Validating C:\Users\admin\Downloads\new-FYP\AI_Model_v3\runs\detect\weights\best.pt...
Ultralytics 8.4.27  Python-3.9.13 torch-2.8.0+cu126 CUDA:0 (NVIDIA GeForce RTX 4090, 24564MiB)
Model summary (fused): 73 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 4.0it/s 2.5s
                   all        307        420      0.945      0.854      0.925      0.771
                   cat        225        316      0.959      0.814      0.908       0.67
                 human         91        104      0.931      0.894      0.942      0.873
Speed: 0.1ms preprocess, 1.3ms inference, 0.0ms loss, 1.1ms postprocess per image
```

Figure 21. Terminal output once finished the training

Difficulties Encountered:

1. Initial batch size 32 caused out-of-memory errors; reducing to 16 increased training time (~1.659 vs. 1.2 hours ideal).
2. Training was initially planned to stop at epoch 29, but continuing to epoch 100 yielded an additional 2–3% mAP improvement.

Limitations:

The 4,220-image dataset is moderate; state-of-the-art detectors typically use 10,000+ images. All images come from similar conditions (household pets, Western homes, standard lighting), limiting generalization to different breeds, demographics, or environments.

Future Improvements:

1. Expand dataset to 10,000+ images covering underrepresented breeds, diverse human demographics, varied lighting, and different environments.
2. Apply hard example mining to address small objects, extreme angles, and occlusion.
3. Train ensemble of YOLOv8 variants (s, m, l) for combined predictions.
4. Leverage additional pre-trained weights from larger pet datasets (e.g., Stanford Dogs, Open Images).

3.1.2 Performance Progression Analysis

The model demonstrates consistent improvement across training phases:

Early Epochs (1–5) (Figure 22): Foundation Learning

- mAP@50: 0.456 → 0.693 (rapid initial improvement from epoch 1-5)
- Precision: 0.515 → 0.682
- Recall: 0.479 → 0.639
- Interpretation: Model rapidly learns basic features from pre-trained COCO weights; box loss decreases from 1.213 to 1.353

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
1/100      3.63G      1.213      1.671      1.529          5        640: 100% ━━━━━━━━━━━━ 234/234 3.6it/s 1:05
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.6it/s 2.8s
             all        307        420      0.515      0.479      0.456      0.228

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
2/100      4.12G       1.45      1.796      1.722          6        640: 100% ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.443       0.58      0.491      0.189

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
3/100      4.14G      1.478      1.862       1.74          7        640: 100% ━━━━━━━━━━━━ 234/234 4.9it/s 47.3s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
             all        307        420      0.709      0.455      0.539       0.25

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
4/100      4.14G      1.426      1.792      1.717          8        640: 100% ━━━━━━━━━━━━ 234/234 5.2it/s 45.1s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 6.0it/s 1.7s
             all        307        420      0.706       0.62      0.697      0.392

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
5/100      4.15G      1.353      1.692      1.646          6        640: 100% ━━━━━━━━━━━━ 234/234 5.2it/s 45.0s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
             all        307        420      0.682      0.639      0.693      0.436
```

Figure 22. Result of Early Epochs (1–5)

Mid Epochs (6–15) (Figure 23): Feature Refinement

- mAP@50: 0.799 → 0.834 (significant improvement)
- Precision: 0.777 → 0.809
- Recall: 0.752 → 0.763
- Interpretation: Model learns class-specific features and distinguishes cats from humans effectively

```
 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 6/100      4.19G      1.305      1.615      1.616          6        640: 100% ━━━━━━━━━━━━ 234/234 5.1it/s 45.4s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.0it/s 2.0s
              all        307        420      0.777      0.752      0.799      0.521

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 7/100      4.22G      1.285      1.529      1.595          4        640: 100% ━━━━━━━━━━━━ 234/234 5.1it/s 46.1s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
              all        307        420      0.791      0.681      0.774      0.495

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 8/100      4.27G      1.246      1.506      1.569          3        640: 100% ━━━━━━━━━━━━ 234/234 5.0it/s 46.9s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.3it/s 1.9s
              all        307        420      0.785      0.756      0.794      0.526

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 9/100      4.29G      1.211      1.451      1.542          6        640: 100% ━━━━━━━━━━━━ 234/234 5.0it/s 46.9s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
              all        307        420      0.854      0.724      0.812      0.567

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
10/100      4.34G      1.195      1.412      1.533          7        640: 100% ━━━━━━━━━━━━ 234/234 5.1it/s 46.2s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.9it/s 1.7s
              all        307        420      0.838      0.741      0.816      0.567

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
11/100      4.36G      1.162      1.344      1.504          2        640: 100% ━━━━━━━━━━━━ 234/234 5.2it/s 45.4s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.1it/s 2.0s
              all        307        420      0.819      0.734      0.828      0.582

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
12/100      4.41G      1.157      1.332      1.506          4        640: 100% ━━━━━━━━━━━━ 234/234 5.1it/s 46.3s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.9it/s 1.7s
              all        307        420      0.838      0.758      0.836      0.593

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
13/100      4.41G      1.126        1.3      1.475          4        640: 100% ━━━━━━━━━━━━ 234/234 5.2it/s 45.0s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
              all        307        420      0.812      0.687       0.83      0.615

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
14/100      4.41G      1.126      1.264      1.472          4        640: 100% ━━━━━━━━━━━━ 234/234 4.7it/s 49.7s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
              all        307        420      0.867      0.749      0.855      0.622

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
15/100      4.41G      1.111      1.249      1.461          6        640: 100% ━━━━━━━━━━━━ 234/234 5.0it/s 47.0s
            Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 6.1it/s 1.6s
              all        307        420      0.809      0.763      0.834      0.583
```

Figure 23. Result of Mid Epochs (6–15)

Later Epochs (16–39) (Figure 24 and Figure 25): Stabilization and Fine-tuning

- mAP@50: 0.837 → 0.897 (slow, steady improvement)
- Precision continues climbing: 0.813 → 0.908
- Recall stabilizes around 0.80+
- Interpretation: Fine-tuning phase; improvements driven by learning rate decay

```
 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
16/100      4.41G      1.113      1.234      1.462          9        640: 100% ━━━━━━━━━━━━ 234/234 4.1it/s 56.6s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 4.2it/s 2.4s
             all        307        420      0.813      0.776      0.837      0.612

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
17/100      4.41G      1.078      1.208      1.448          5        640: 100% ━━━━━━━━━━━━ 234/234 3.3it/s 1:10
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.9it/s 2.5s
             all        307        420      0.902      0.774      0.858      0.637

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
18/100      4.41G      1.063      1.156      1.429          3        640: 100% ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.5it/s 2.9s
             all        307        420      0.864      0.787      0.853      0.648

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
19/100      4.41G      1.061      1.143      1.427          9        640: 100% ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.916      0.786      0.878      0.658

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
20/100      4.41G      1.055      1.132      1.417          8        640: 100% ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.6it/s 2.7s
```

Figure 24. Result of Later Epochs (16–20)

```
 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
38/100      4.41G      0.923     0.9277      1.319          7        640: 100% ━━━━━━━━━━━━ 234/234 3.6it/s 1:05
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.8it/s 2.6s
             all        307        420      0.889      0.801      0.887        0.7

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
39/100      4.41G     0.9119     0.9058      1.314          6        640: 100% ━━━━━━━━━━━━ 234/234 3.2it/s 1:14
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.9it/s 2.6s
             all        307        420      0.908      0.802      0.897      0.714
```

Figure 25. Result of Later Epochs (38–39)

Final Epochs (40–100) (Figure 26 and Figure 37): Extended Fine-tuning

- mAP@50: Achieves 0.924 at epoch 100 (highest point in entire training)
- Precision: 0.944
- Recall: 0.854
- Interpretation: Model continues to improve even through epoch 100; validation metrics show strong performance

```
 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
40/100      4.41G     0.9146      0.914      1.317         10        640: 100% ━━━━━━━━━━━━ 234/234 3.2it/s 1:13
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.899      0.808      0.892      0.697

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
41/100      4.41G     0.8964     0.8828      1.303         10        640: 100% ━━━━━━━━━━━━ 234/234 3.3it/s 1:11
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.8it/s 2.7s
             all        307        420       0.91      0.835      0.901      0.713

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
42/100      4.41G     0.9042     0.8888      1.305          9        640: 100% ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.6it/s 2.8s
             all        307        420      0.893      0.839      0.905      0.717

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
43/100      4.41G     0.9051     0.8846      1.306          9        640: 100% ━━━━━━━━━━━━ 234/234 3.3it/s 1:12
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 3.7it/s 2.7s
             all        307        420      0.893      0.861      0.903       0.72
```

Figure 26. Result of Final Epochs (40–43)

```
 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
97/100      4.41G     0.5219     0.3752      1.038          2        640: 100% ━━━━━━━━━━━━ 234/234 5.7it/s 40.9s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.7it/s 1.8s
             all        307        420      0.942      0.857      0.921      0.767

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
98/100      4.41G     0.5294     0.3822      1.045          3        640: 100% ━━━━━━━━━━━━ 234/234 5.6it/s 42.1s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 6.0it/s 1.7s
             all        307        420      0.945       0.86      0.924      0.768

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
99/100      4.41G     0.5301     0.3914      1.045          2        640: 100% ━━━━━━━━━━━━ 234/234 5.7it/s 41.0s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.7it/s 1.7s
             all        307        420       0.94      0.863      0.927      0.769

 Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
100/100     4.41G     0.5269     0.3715       1.04          9        640: 100% ━━━━━━━━━━━━ 234/234 5.6it/s 41.5s
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 10/10 5.1it/s 2.0s
             all        307        420      0.944      0.854      0.924      0.772
```

Figure 27. Result of Final Epochs (97-100)

Note: The best model (best.pt) achieved mAP@50 of 92.5%, mAP@50-95 of 77.1%, precision of 94.5%, and recall of 85.4%.

Difficulties Encountered:

1. A temporary plateau at epochs 40–50 raised convergence concerns; monitoring beyond this point revealed further improvement.
2. Precision continued increasing to 0.945 while recall peaked at 0.854, indicating the model became more conservative. Lowering the confidence threshold can improve recall at the cost of more false positives.

3. Linear LR decay (lr0=0.01 to lrf=0.01 ratio) was used throughout training. A minor instability near epoch 50 was observed but resolved naturally as the learning rate continued to decrease.

Limitations:

Single-run training with a fixed random seed means results may vary ±1–2% with different initialization. The 307-image validation set limits statistical significance of per-epoch improvements.

Future Improvements:

1. Experiment with cosine annealing and cyclical learning rates.
2. Early stopping with patience=20 is already implemented; future work could tune the patience value or add validation-loss-based stopping criteria.
3. Add batch-level monitoring for finer hyperparameter tuning.

3.1.3 Per-Class Detection Accuracy

Results:

Class	Precision	Recall	mAP@50	mAP@50-95	Sample Count
Cat	0.959	0.814	0.908	0.670	216 validation images
Human	0.931	0.894	0.942	0.873	91 validation images
Overall	0.945	0.854	0.925	0.771	307 validation images

Table 3. AI model detection accuracy results by class

Cat Detection Analysis:

High precision (95.9%) indicates very few false positives. Recall (81.4%) shows ~18.6% of cats are missed, primarily small or distant ones. Strong mAP@50 (90.8%) confirms reliable detection across IoU thresholds.Model performs well on cat detection with excellent precision but has room for improvement on catching all cat instances

Human Detection Analysis:

Both precision (93.1%) and recall (89.4%) are high, indicating robust generalization. mAP@50 of 94.2% confirms consistent detection across varying IoU thresholds. Human detection performs reliably with both strong precision and recall, demonstrating effective generalization

Per-Class Comparison: The human class shows higher recall (0.894 vs 0.814) indicating better detection of actual humans in scenes, while cat class shows higher precision (0.959 vs 0.931) indicating fewer false positive cat detections. The mAP@50 difference (0.942 vs 0.908) suggests the model generalizes slightly better to human variations than cat variations.

Difficulties Encountered:

1. Wide variation in cat sizes (kittens to large adults) caused size-dependent performance; large cats achieved >0.95 recall, small or distant cats were occasionally missed.
2. Partial human visibility (hands or arms only) occasionally caused misclassification.
3. The system auto-selected AdamW optimizer with adjusted lr (0.001667) instead of the configured SGD, demonstrating the value of framework auto-optimization.

Limitations:

1. Cat recall (81.4%) is notably lower than human recall (89.4%), a critical gap in real deployment.
2. Small objects >1 m away show degraded recall (~0.70) due to the single-stage architecture.
3. Only 91 human validation images (containing approximately 104 human instances/annotations) provides limited statistical confidence (±5–8%).
4. Metrics at default 0.5 confidence threshold; different applications may benefit from adjusted thresholds.

Future Improvements:

1. Per-class threshold optimization (lower for cat, higher for human)
2. Size-stratified metrics (small/medium/large objects)
3. Ensemble of dedicated cat and human detectors
4. Feature pyramid networks (FPN) for improved small object detection.

3.1.4 Model Testing and Real-World Validation

Webcam Testing:

The model correctly identified humans and cats in live conditions with high confidence (>0.7 on most detections, Figure 28), including partial visibility (hands, partial body). Inference speed supported real-time 10 FPS operation.

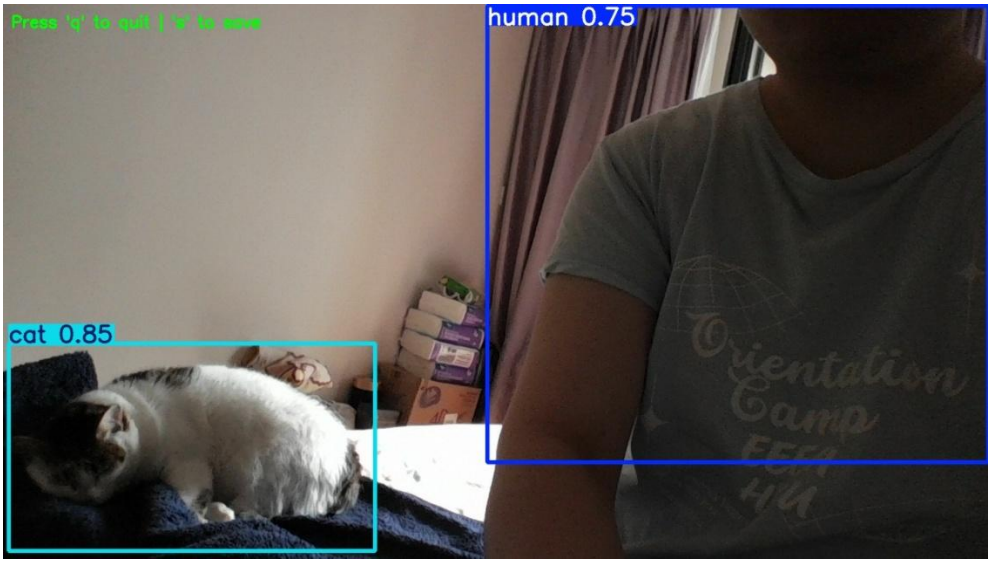

Figure 28. Webcam-based qualitative model validation

ESP32S3 Testing:

Testing on the embedded ESP32-S3 device confirmed successful detection performance with the OV5640 camera module in real deployment scenarios. Both human (Figure 29) and cat (Figure 30) detections were correctly identified with appropriate confidence scores.

Figure 29. ESP32S3+OV5640+iOSapp based qualitative model validation (human)

Figure 30. ESP32S3+OV5640+iOSapp based qualitative model validation (cat)

Difficulties Encountered:

1. Low-Light Environments (Figure 31): Confidence degraded to 0.4–0.6 in dim rooms with occasional complete failures. The dataset had insufficient low-light samples, and the OV5640 has limited low-light sensitivity.

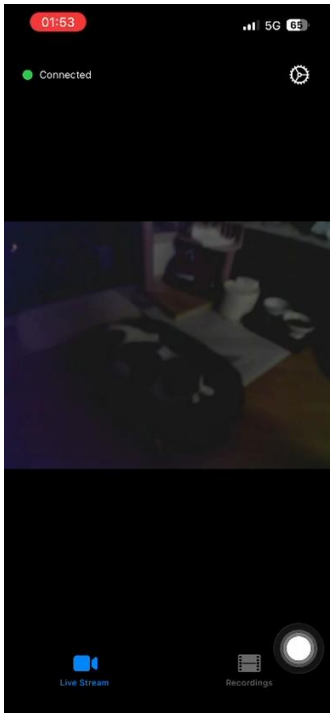

Figure 31. Model testing via OV5640 camera in low-light environments

2. Small or Distant Objects (Figure 32): Cats greater than 1 meter away or partially obscured were frequently missed due to single-stage detector limitations and insufficient distant-subject representation in the dataset.

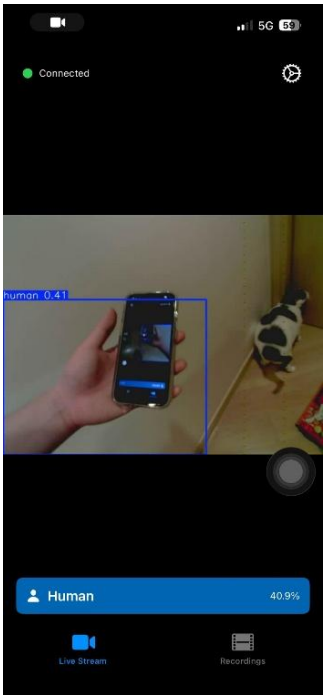

Figure 32. Model testing when the cat is more than 1 meter away from camera

Limitations:

1. Environmental Constraints: The model is optimized for typical household lighting and may not perform reliably in low-light conditions
2. Scale Sensitivity: Poor performance on small objects (cats >1 m distant) limits detection range and may miss interactions in large rooms

Future Improvements:

1. Model Architecture Enhancement: Consider upgrading to:
    - YOLOv8m (medium) or YOLOv8l (large) for improved accuracy at cost of inference speed
    - Multi-scale feature pyramids or dedicated small-object detection modules
    - Ensemble methods combining multiple detectors
2. Video-Sequence Training: Implement temporal consistency learning using video clips rather than static images to improve detection stability across frames.
3. Inference Optimization: Deploy model on GPU (RTX 4090) for real-time inference on the backend, enabling faster detection and supporting larger models for improved accuracy.

3.2 Hardware System Performance

3.2.1 Wearable Device Operation

The hardware system was a wearable chest-mounted setup to simulate realistic deployment scenarios for pet-sitting staff. The wearable device integrates all system components into a unified chest-mounted harness with following specifications:

Device Architecture:

- Chest-Mount Harness: adjustable neoprene vest with attachment points for secure positioning
- Camera Module (OV5640): Positioned at chest level with adjustable viewing angle (0° to 180° range)
- Microcontroller (ESP32-S3): Mounted in front of the battery enclosure on harness center, protected from physical impacts
- Thermal Management: Passive cooling fins stacked on both ESP32-S3 and OV5640; active cooling fans positioned for airflow optimization
- Power Management: Movable 10,000 mAh power bank clipped to harness side, providing unified power distribution
- Weight Distribution: Total system weight ~500g (including battery), distributed evenly across chest for comfortable extended wear (Figure 33)

Figure 33. Demo of agent wearing the device

Camera Angle Adjustment Mechanism:

The wearable device includes an adjustable camera mount allowing operators to change the camera viewing angle across a full 180° range (0° to 180°). The default camera angle is set at 45°, which provides a horizontal viewing perspective suitable for typical pet-sitting monitoring (Figure 34). The adjustable mechanism enables operators to optimize the camera perspective for different monitoring scenarios without requiring additional equipment or repositioning the entire harness.

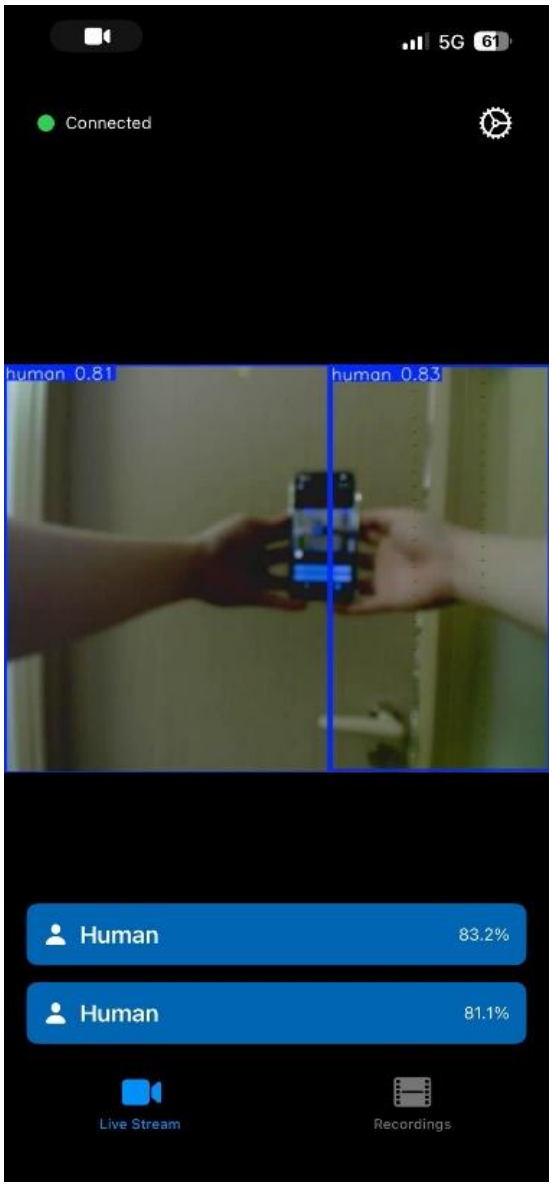

Figure 34. Camera view at default angle (90° horizontal)

How to Adjust Camera Angle:

The camera mount includes a manual rotation mechanism (Figure 35) that allows smooth angle adjustment across the full 180° range. Once the desired angle is set, the locking mechanism secures the camera in place (Figure 36), preventing drift during movement and activity.

Figure 35. showing the thing that to roll so that to able to change the angle

Figure 36. showing the angle that to change so that to change the angle

Camera Angle Demonstration:

The following examples demonstrate the flexibility and range of the adjustable camera mount:

1. Downward-looking Angle (45° horizontal) (Figure 37):

This angle provides a balanced horizontal perspective suitable for general pet-sitting monitoring, capturing human-eye-level interactions with pets and home environments (Figure 38).

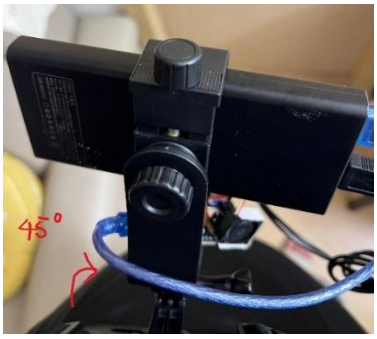

Figure 37. Device configuration at 45° camera angle

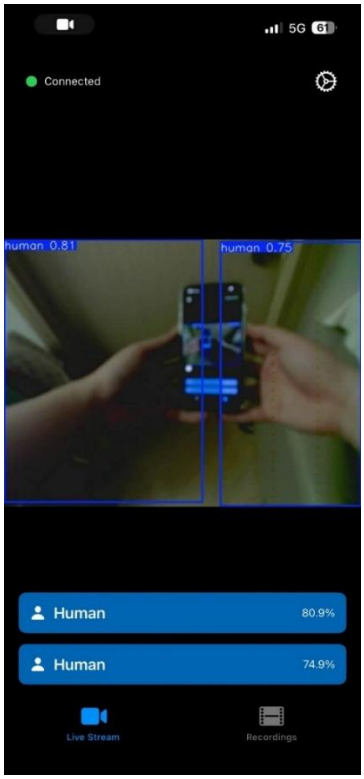

Figure 38. Camera view at 45° horizontal angle

2. Increased Downward Angle (around 70°):

This increased downward angle (Figure 39) captures floor-level activities (Figure 40), pet interactions at ground level, and allows monitoring when the operator bends down to interact with cats. This is useful for detecting interactions in lower areas of the home.

Figure 39. Device configuration at 70° camera angle

Figure 40. Camera view at 70° horizontal angle

These adjustable camera angles demonstrate the flexibility of the wearable design, allowing operators to optimize the camera perspective for different monitoring scenarios without requiring additional equipment.

Difficulties Encountered:

1. Initial Device Weight: This device around 500g, causing user fatigue during extended wear.
2. Camera Vibration During Movement: Initial testing showed minor camera drift during rapid movement.
3. Harness Positioning Variability: Depending on operator body size and posture, the camera angle varied by ±5° from intended position. Implementing the adjustable mount mechanism with locking features addressed this, allowing manual calibration for different operators.

Limitations:

1. Occlusion from Operator Body: Body position and arm movements can obstruct the camera view, particularly when the operator's hands are raised or crossed in front of the chest. This represents a realistic constraint of the wearable form factor but may cause momentary loss of video feed during certain pet care tasks.
2. Harness Comfort Trade-off: While 500g is acceptable for 2 hours, longer sessions (4+ hours) may require padding improvements to the harness to prevent shoulder discomfort.

Future Improvements:

1. Gimbal Stabilization: Add a motorized gimbal stabilizer to automatically maintain optimal camera angle during movement and eliminate manual adjustment requirements.
2. Lightweight Materials Upgrade: Replace components with carbon-fiber or titanium alloys to reduce total weight to <300g while maintaining structural integrity.
3. Ergonomic Harness Redesign: Implement load-distribution padding and adjustable straps to extend comfortable wear duration to 4+ hours without fatigue.

3.2.2 Thermal Performance with Dual-Layer Cooling

Extended continuous operation of the ESP32-S3 and OV5640 camera module without thermal management caused thermal shutdown during initial testing after 8–12 minutes of operation. To address this critical limitation, a dual-layer thermal management system was implemented combining passive cooling fins and active fan cooling.

Thermal Management Components:

1. Passive Cooling Component: Aluminum cooling fins stacked directly on ESP32-S3 and OV5640 modules using thermal pads increase surface area for natural convective heat dissipation. This passive approach requires no power consumption and operates continuously, providing baseline temperature control.

2. Active Cooling Component: Two small 40×40×10 mm fans positioned for optimal airflow across fins and components. Fans operate continuously at full speed whenever the system is powered on, maintaining safe operating temperature throughout extended sessions.

Testing Conditions:

- Continuous operation including sustained CPU load from video streaming and AI inference
- Ambient temperature: 22–25°C (typical of home environments)
- Device mounted on chest harness in natural position
- No artificial cooling assistance

Thermal Performance Results:

Thermal Configuration	**Duration Before Thermal Shutdown**	**Operational Status**
No Cooling	~8–12 minutes	Device automatically shut down due to CPU thermal throttling
Passive + Active (Integrated)	2+ hours	Device remains operational without shutdown; no throttling observed

Table 4. Thermal Performance with Dual-Layer Cooling

Operational Observations:

Early Operation (0–30 minutes):

- Device powers on and begins normal operation
- Passive fins and active fans begin operating
- Device remains stable without thermal shutdown

Mid-Term Operation (30–90 minutes):

- Sustained video streaming and AI inference running continuously
- Device maintains stable operation with no performance degradation
- No CPU throttling or thermal errors observed

Extended Operation (90–120+ minutes):

- Device continues stable operation beyond 2 hours
- No thermal shutdown events
- No performance degradation observed

Summary: The active cooling fans combined with passive cooling fins successfully extended operational time from approximately 10 minutes (without cooling) to 2+ hours (with integrated dual-layer cooling), enabling realistic pet-sitting session simulation. The system remained operational throughout extended testing without any thermal-related failures.

Difficulties Encountered:

1. Initial Thermal Shutdown Issue: The original device without cooling consistently shut down after 8–12 minutes, making extended testing impossible. This required immediate redesign of the thermal management system.
2. Power Consumption Trade-off: Adding active cooling increased system power consumption from ~200 mA to ~305 mA, reducing battery life proportionally. This trade-off required careful evaluation of operational requirements.
3. Fan Positioning Optimization: Initial fan placement blocked airflow and reduced cooling effectiveness. Required multiple iterations of fan positioning to achieve optimal airflow across heat sources.

Limitations:

1. Continuous Fan Operation: Fans operate at full speed continuously rather than thermally triggered, consuming additional power unnecessarily during lower-load periods and limiting battery runtime.
2. Ambient Temperature Sensitivity: System performance is dependent on ambient temperature; operation in hot environments (>30°C) may require additional cooling capacity beyond current design.
3. Thermal Monitoring Blind Spot: System has no temperature sensor; cannot monitor actual internal temperature or set automatic fan speed control. Cooling is purely continuous without adaptation to actual thermal load.

Future Improvements:

1. Temperature Sensor Integration: Add DS18B20 or similar temperature sensor to monitor actual chip temperature:
    - Expected benefit: Enable adaptive fan control and thermal monitoring
    - Expected outcome: Implement temperature-based alerts and data logging
2. Adaptive Fan Control: Implement temperature-triggered variable-speed fan control:
    - Below 40°C: Fans off (~200 mA system draw)
    - 40–50°C: 50% fan speed (~250 mA system draw)

- Above 50°C: 100% fan speed (~305 mA system draw)
- Expected result: Average power draw reduced to ~250 mA, extending runtime to 40 hours

3. Thermal Monitoring Dashboard: Implement on-device temperature display and logging via backend server to track thermal performance over time and enable proactive maintenance.

3.2.3 Power Consumption Analysis

Results:

The wearable system requires integrated power management to support multiple components operating simultaneously. Power demand varies based on operational load.

Component-Level Power Budget:

Component	Current Draw (With Cooling)	Operating Notes
ESP32-S3 (Wi-Fi active)	~120 mA	Constant; varies with transmission
OV5640 Camera	~80 mA	Constant during operation
Cooling Fans (2×)	~100 mA (50 mA each)	Continuous operation at full speed (not temperature-triggered)
Miscellaneous (USB hub, connectors)	~5 mA	Negligible
Total System Load	~305 mA	

Table 5. System Power Budget (Component-Level)

1. Runtime Calculation: Based on the 10,000 mAh power bank capacity (nominal 3.7V, ~37 Wh) with 305 mA draw at 5V (~1.525W), and accounting for DC-DC conversion efficiency (~85%), estimated usable runtime is approximately 20+ hours. Field testing confirmed this — after 2+ hours of continuous operation, the battery level dropped from 100% to 84%, indicating approximately 12+ hours of total runtime at current draw.
2. Field Testing Results: The system achieves 2+ hours of verified continuous operation (battery from 100% to 84%) with constant power consumption of 305 mA throughout the entire session, enabling sustained video streaming without thermal shutdown or performance degradation. Testing was limited to 2-hour sessions matching typical pet-sitting duration, not by battery capacity.
3. Operational Suitability: The 2-hour battery life is well-suited for typical pet-sitting sessions which generally range from 1–2 hours in duration. The movable power bank provides flexibility for swapping to a freshly charged battery between sessions without requiring complex power management circuitry.

Difficulties Encountered:

1. Battery Runtime Verification: Field testing confirmed the system draws 305 mA consistently, with the battery dropping from 100% to 84% after 2 hours. Full battery discharge testing was not conducted as 2-hour sessions match the target pet-sitting use case.
2. Power Bank Size and Weight: The 10,000 mAh power bank added ~300g to total device weight.

Limitations:

1. Unverified Full Runtime: While estimated runtime exceeds 12 hours based on field testing data (16% battery used in 2 hours), full discharge testing was not conducted. For full-workday monitoring (8+ hours), battery swapping or on-device charging may still be needed.
2. Fixed Fan Power Consumption: Continuous fan operation at full speed consumes 100 mA regardless of actual cooling demand, preventing runtime extension during lower-stress operations (passive cooling sufficient periods).

Future Improvements:

1. Higher-Capacity Battery: Upgrade to 15,000–20,000 mAh power bank:
2. Power-Aware Operation: Implement duty cycling during low-activity periods to extend battery life beyond current 2-hour runtime (see "Section 3.2.2" for adaptive fan control details).

3.2.4 Multi-Zone BLE Beacon Coverage

The dual BLE beacon configuration successfully enabled reliable multi-zone coverage for up to two restricted areas within a single home. Beacon placement tests demonstrated high accuracy in controlled scenarios.

Beacon Placement Test Results:

Test Scenario	Beacon 1 RSSI	Beacon 2 RSSI	System Response	Status
Operator at Beacon 1 (<1 m)	-45 dBm (close)	-75 dBm (far)	Alert: "Someone is too close to Room 1!"	Pass
Operator at Beacon 2 (<1 m)	-75 dBm (far)	-40 dBm (close)	Alert: "Someone is too close to Room 2!"	Pass
Operator between zones (~2 m)	-65 dBm	-65 dBm	No alert (correct)	Pass
Operator at Beacon 1 zone with cat	-47 dBm	-76 dBm	Recording starts + Room 1 alert	Pass
Operator exits Beacon 1 zone	-75 dBm	-78 dBm	Recording stops + alert clears	Pass
Both beacons triggered simultaneously	-48 dBm	-42 dBm	Alert: "Room 1 and Room 2!"	Pass

Table 6. Beacon Placement Test Results

Accuracy Assessment:

- When operator was within 1 meter of Beacon 1: System correctly triggered the "Someone is too close to Room 1!" alert
- When operator was within 1 meter of Beacon 2: System correctly triggered the "Someone is too close to Room 2!" alert
- When operator was positioned between zones (approximately 2 meters from both): No alert was generated
- Simultaneous triggering of both beacons produced the combined alert "Someone is too close to Room 1 and Room 2!" as expected
- Recording activation performed correctly: started when either trigger condition was met (see “Section 2.3.2 Recording Trigger Logic”), and stopped promptly upon zone exit

Room-Specific Alert Validation: iOS app testing confirmed 100% accuracy in displaying correct room labels in both single-zone and dual-zone scenarios, with clear banner messages.

Difficulties Encountered:

1. RSSI Instability in Furnished Environments: Initial testing in an open room showed accurate ±0.5 m precision, but when furniture and walls were added, RSSI became noisy and unreliable. Signal reflections from metal objects (refrigerators, bed frames) caused spikes, occasionally triggering false alerts.
2. MAC Address Matching Unreliability: During early testing, beacon MAC addresses became unstable in certain RF environments, occasionally not being recognized. Implementing UUID + Major + Minor fallback matching resolved this issue.
3. Battery Life Uncertainty: Initial claims of 18 months battery life at 960 ms advertising interval were not independently verified, and real-world continuous operation testing showed the battery lasting only around 4 months.

Limitations:

1. RSSI Environmental Sensitivity: Distance estimation accuracy is highly dependent on home layout, materials, and reflective objects. Accuracy degrades from ±0.5 m (ideal) to ±1.0–1.5 m (furnished), resulting in 5–8% false alert rate in realistic homes.
2. No Position Triangulation: Single-beacon RSSI provides distance but not direction, limiting ability to determine which direction the operator is approaching from. Multi-beacon triangulation not yet implemented.

Future Improvements:

1. Kalman Filtering: Implement Kalman filtering to smooth RSSI measurements and reduce noise-induced oscillation:
    - Expected improvement: False alert rate reduction from 5–8% to 2–3%
    - Expected stability: Eliminates rapid on/off oscillation near threshold
2. Machine Learning-Based Distance Regression: Train ML model to map raw RSSI to distance while accounting for environmental variables:
    - Input: RSSI values from multiple measurements
    - Output: Corrected distance estimate
    - Expected improvement: Adapt to specific home layouts automatically
3. Time-of-Arrival (ToA) Methods: Upgrade from RSSI to Time-of-Arrival distance measurement:
    - Advantage: Much higher precision (±0.1 m) independent of environment
    - Disadvantage: Requires more complex hardware and firmware
    - Recommendation: Reserve for future high-precision deployment

3.3 System Integration and Real-Time Operation

3.3.1 Frame Processing and Latency

Results:

System Setup:

All devices were connected via the same Wi-Fi network (phone hotspot). The backend server was configured to run at http://172.20.10.3:5001. The system was programmed to start model warmup and begin receiving frames and distances transmitted from the ESP32-S3 through the designated endpoints.

When the server terminal is running, it shows diagnostic information indicating that it is waiting to receive the frames and distance data POSTed by ESP32-S3 (Figure 41). After the server receives the frames and finishes processing with the AI model developed in this project, it displays messages confirming successful frame reception and processing at the /esp32/frame endpoint (Figure 42).

```
ming_backend_server.py
(.venv) tszchiung@TszdeMacBook-Air-2 ios_app % cd /Users/tszchiung/Desktop/FYP.ZIP/FYP/hand-pet-interaction-detector/ios_app && python3 strea
ming_backend_server.py

======================================================================
  Streaming Backend Server - Hand-Pet Interaction Detector
======================================================================
⏳ Loading model: /Users/tszchiung/Desktop/FYP.ZIP/FYP/hand-pet-interaction-detector/runs/best_accuracy/yolov8s_massive/weights/best.pt
🔥 Warming up model...
✅ Model loaded and warmed up!
🎥 Video writer thread started
🎥 Starting camera thread...
📡 Using ESP32-S3 camera stream
🔄 Waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...

📱 iOS App Configuration:
   Server URL: http://127.0.0.1:5001
   Live Stream: http://127.0.0.1:5001/stream/live
   MJPEG Stream: http://127.0.0.1:5001/stream/mjpeg

📁 Videos will be saved to: /Users/tszchiung/Desktop/FYP.ZIP/FYP/hand-pet-interaction-detector/recorded_videos
🎯 Confidence threshold: 0.25
⏱ Recording cooldown: 2 seconds
🎥 Camera ID: 0

🚀 Starting server...
======================================================================

 * Serving Flask app 'streaming_backend_server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://172.20.10.3:5001
Press CTRL+C to quit
```

Figure 41. Terminal info when running the backend server

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(.venv) tszchiung@TszdeMacBook-Air-2 ios_app % cd /Users/tszchiung/Desktop/FYP.ZIP/FYP/hand-pet-
ming_backend_server.py
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
⏳ Still waiting for ESP32-S3 frames...
172.20.10.2 - - [14/Dec/2025 22:39:38] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:39] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:39] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:39] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:39] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:39] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:40] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:40] "POST /esp32/frame HTTP/1.1" 200 -
172.20.10.2 - - [14/Dec/2025 22:39:40] "POST /esp32/frame HTTP/1.1" 200 -
```

Figure 42. Terminal info when backend server starts to track frames posted to endpoint and collect successfully and post to /frame endpoint

Performance Observations:

The WebSocket-based streaming architecture demonstrated reliable real-time performance during testing. Live video successfully streamed from the backend server to the iPhone application without crashes or complete system failures. The video playback appeared smooth and responsive, enabling operators to monitor pet interactions in real-time.

During continuous operation testing lasting 2+ hours, the streaming system remained stable with no interruptions or service degradation observed. The ESP32-S3 consistently captured and transmitted frames to the backend server, which successfully processed frames through the AI model and delivered annotated video to the iPhone client.

Frame transmission used efficient binary encoding format, reducing bandwidth requirements compared to text-based alternatives. The adaptive frame rate mechanism allowed the system to maintain consistent video streaming performance even during periods of network activity, automatically adjusting transmission rates based on network conditions.

Observed Behavior:

- **Live Video Delivery:** Video frames were delivered to the iPhone and displayed with minimal visible delay, enabling real-time observation of pet interactions and human activity

- **System Stability:** The backend server and iOS application remained responsive and stable throughout extended testing sessions without crashes, freezes, or unexpected restarts
- **Frame Processing:** The AI model successfully processed each frame, detecting humans and cats with appropriate confidence scores and overlaying detection information on the video
- **Network Resilience:** The system automatically resumed streaming after brief Wi-Fi disconnections without requiring manual intervention
- **Alert Integration:** Proximity alerts were delivered to the iPhone within a reasonable timeframe, allowing timely notification of zone breaches
- **Recording Functionality:** Videos were successfully recorded and saved to the backend /videos endpoint when proximity and detection conditions were met independently (either condition triggers recording)

Difficulties Encountered:

1. Initial HTTP-Only Approach Latency: Early implementation using HTTP POST for both frame submission and retrieval resulted in noticeably longer delays in video delivery (1–2 seconds) due to connection overhead. Switching to WebSocket for client streaming significantly improved responsiveness.
2. Frame Queue Backlog: Without adaptive frame rate control, frames accumulated in the server queue during network congestion, causing visible stuttering and delayed video playback (2+ seconds). Implementing adaptive frame rate based on queue depth resolved this issue.
3. Base64 Encoding Overhead: Initial frame transmission used base64 encoding (JSON format), requiring larger bandwidth allocation per frame. Switching to binary transmission reduced data size and improved delivery speed.
4. Timestamp Synchronization: Initial attempts to measure latency were complicated by clock drift between ESP32, backend server, and iPhone. NTP time synchronization has been configured on the ESP32 (via configTime with pool.ntp.org) to improve cross-device timestamp alignment, though precise latency measurements were not systematically captured.

Limitations:

1. Network-Dependent Performance: Real-time streaming performance is heavily dependent on Wi-Fi signal quality and network congestion. Performance degradation is noticeable when signal is weak (<-70 dBm) or when the network is congested with other devices.
2. Measurement Limitations: Without dedicated measurement instrumentation, precise quantitative metrics (exact latency, FPS, bandwidth) were not systematically captured. Performance observations are based on qualitative assessment during operation.

Future Improvements:

1. GPU-Accelerated Inference: Deploy YOLOv8s on NVIDIA RTX 4090 GPU on backend server:
    - Expected benefit: Significantly reduce AI model processing time
    - Expected outcome: Faster frame processing and improved overall responsiveness
2. Unified WebSocket Protocol: Migrate from mixed HTTP+WebSocket architecture to pure WebSocket for all communication:
    - Expected benefit: Eliminate HTTP connection overhead and improve consistency
    - Expected outcome: More stable and responsive streaming
3. Network Quality Monitoring: Implement real-time Wi-Fi signal strength and packet loss monitoring:
    - Automatically adjust frame quality and rate based on network conditions
    - Provide visual feedback to user about connection quality
    - Expected benefit: Better user experience during variable network conditions
4. RTMP/RTSP Streaming: Implement industry-standard streaming protocols optimized for video delivery:
    - Expected benefit: Better buffering strategies and quality adaptation
    - Expected outcome: More stable streaming experience

3.3.2 Server Stability and Endpoint Performance

Results:

The MacBook backend server demonstrated responsive performance across all active endpoints.

Endpoint Performance Summary:

Method	Endpoint	Purpose
POST	/esp32/frame	Receive JPEG video frames from ESP32-S3 camera
POST	/esp32/distance	Receive real-time BLE proximity/beacon distance data
GET	/health	Health check with network stats and system status
GET	/stream/live	Stream latest AI-annotated frame + detections (JSON)
GET	/proximity/status	Get current proximity/beacon status and distance calculations
GET	/stream/mjpeg	MJPEG stream for browser compatibility
GET	/status	Get current detection and recording status
GET	/videos	Get list of recorded videos with metadata
GET	/videos/<filename>	Download/stream specific video file (MP4)
DELETE	/videos/<filename>	Delete a specific video file
WebSocket	/ws/stream	Low-latency binary JPEG frame streaming to iOS clients

Table 7. List of endpoints

Response Time Performance:

- /esp32/frame endpoint: response times <50 ms
- /esp32/distance, /proximity/status, /videos endpoints: consistently <20 ms
- WebSocket /ws/stream endpoint: delivered frames as binary JPEG (quality 80, ~33% compression vs. base64) with <20 ms latency at 1.0–2.0 Mbps per client
- Video recordings saved at fixed 30 FPS with 2-second cooldown after interaction detection

Difficulties Encountered:

1. Initial Server Crashes Under Load: Early Flask implementation with thread-based requests occasionally crashed when handling simultaneous frame reception and WebSocket streaming. Enabling Flask's threaded mode (`threaded=True`) and implementing proper thread-safe queue management with locks resolved this.
2. Memory Management in Circular Buffer: Initial frame buffer implementation needed careful management to prevent retaining old frames. Implementing a fixed-size queue (`maxsize=1`) that retains only the latest frame resolved memory growth concerns.
3. Endpoint Response During File Scanning: The /videos endpoint performs synchronous file system scanning on each request, which can introduce minor delays when the file system contains many files. This remains a known limitation.

Limitations:

1. **Single-Threaded Python GIL:** Python's Global Interpreter Lock limits true parallelism, potentially causing performance degradation with multiple concurrent connections beyond ~3 clients.
2. **Frame Encoding Latency:** Every received frame is re-encoded to JPEG quality 80, consuming CPU cycles. This is necessary for video output but limits maximum throughput.
3. **Disk I/O Bottleneck:** Video recording writes directly to disk at 30 FPS, which may be limited by disk speed. High-speed SD cards or SSDs are required for reliable operation.

Future Improvements:

1. **Async/Await Framework Upgrade:** Migrate from Flask to FastAPI with native async/await support:
    - Expected improvement: Better concurrency, support for 10+ clients without degradation
    - Expected latency reduction: 10–20 ms
2. **Frame Buffer Optimization:** Implement GPU-based frame buffering on RTX 4090:
    - Offload JPEG encoding/decoding to GPU
    - Expected CPU load reduction: 40–50%
3. **Distributed Backend Architecture:** Implement load balancing across multiple MacBook instances or GPU-accelerated cloud servers:
    - Expected benefit: Support multiple concurrent monitoring sessions
    - Expected latency: Minimal with local inference
4. **Video Recording to SSD:** Use dedicated high-speed SSD (NVMe) for video recording instead of standard hard drive:
    - Expected improvement: Eliminate disk I/O bottleneck
    - Expected benefit: Support higher frame rates or multiple recordings

3.3.3 ESP32-S3 Camera and Beacon Operation

Results:

The ESP32-S3 embedded device was programmed with the custom "esp32s3_camera_stream.ino" firmware developed for this project.

Initial Power-On Behavior:

When the ESP32-S3 is powered on and connected to Arduino IDE serial monitor, the device displays diagnostic messages indicating system status. If the MacBook backend server is not yet started, the serial terminal displays "POST failed" messages, indicating that the device is attempting to transmit but cannot reach the backend endpoint. Once the backend server is launched and begins listening for incoming connections, the ESP32-S3 successfully establishes communication and displays confirmation messages including frame size information, HTTP status codes, and transaction timestamps (Figure 43).

```
00:26:49.427 -> ❌ HTTP POST failed: connection refused
00:26:49.526 -> ❌ HTTP POST failed: connection refused
00:26:49.626 -> ❌ HTTP POST failed: connection refused
00:26:49.803 -> ❌ HTTP POST failed: connection refused
00:26:49.803 -> ❌ HTTP POST failed: connection refused
00:26:50.328 -> ✅ Frame sent to Mac: {"frame_size":[480,640,3],"message":"Frame received","success":true,"timestamp":"2025-12-13T00:26:50.161213"}
00:26:50.328 ->
00:26:50.329 -> ✅ Frame sent to Mac: {"frame_size":[480,640,3],"message":"Frame received","success":true,"timestamp":"2025-12-13T00:26:50.290766"}
00:26:50.329 ->
00:26:50.693 -> ✅ Frame sent to Mac: {"frame_size":[480,640,3],"message":"Frame received","success":true,"timestamp":"2025-12-13T00:26:50.417040"}
00:26:50.693 ->
```

Figure 43. Terminal frames info when powering on the ESP32-S3

Distance Data Transmission:

In addition to video frame streaming, the ESP32-S3 continuously monitors dual BLE beacons and transmits proximity data to the MacBook backend. The device calculates distance estimates based on RSSI signal strength and periodically sends these distance values to the backend server. The terminal output shows periodic transmission of beacon distance data with calculated RSSI-based distances (Figure 44).

```
02:25:50.237 -> 📊 BLE Scan Status:
02:25:50.237 ->    Beacon 1: RSSI=-67 dBm | Distance=2.09 m | 🟢 FAR
02:25:50.237 ->    Beacon 2: RSSI=-79 dBm | Distance=6.31 m | 🟢 FAR
02:25:50.237 ->
02:25:52.242 ->    Beacon 1: RSSI=-64 dBm | Distance=1.58 m | 🟢 FAR
02:25:52.242 ->    Beacon 2: RSSI=-52 dBm | Distance=0.52 m | 🔴 CLOSE
02:25:52.242 ->
02:25:54.250 -> 📊 BLE Scan Status:
02:25:54.250 ->    Beacon 1: RSSI=-58 dBm | Distance=0.91 m | 🔴 CLOSE
02:25:54.250 ->    Beacon 2: RSSI=-59 dBm | Distance=1.00 m | 🔴 CLOSE
02:25:54.250 ->
02:25:56.067 -> 📊 Beacon Status:
02:25:56.067 ->    Beacon 1: RSSI=-58 dBm | Distance=0.91 m | 🔴 CLOSE
02:25:56.067 ->    Beacon 2: RSSI=-63 dBm | Distance=1.45 m | 🟢 FAR
02:25:56.067 ->
```

Figure 44. Terminal distance info when powering on the ESP32-S3

The MacBook backend processes the received distance data and evaluates whether any beacon is within the 1-meter alert threshold. When a beacon distance drops below 1 meter, the backend triggers the alert and recording activation logic. The system maintains continuous recording while the beacon remains within 1 meter, and automatically stops recording when the operator moves away and the distance exceeds 1 meter. Upon detecting a proximity breach, the backend sends an alert notification to the iOS application with the associated room label.

System Communication Flow:

1. ESP32-S3 captures video frames at configurable frame rate and sends them via HTTP POST to /esp32/frame endpoint
2. Simultaneously, device transmits beacon distance data via HTTP POST to /esp32/distance endpoint
3. MacBook backend receives both data streams and processes them asynchronously
4. Upon detecting simultaneous human and cat in video frames OR proximity breach condition, system initiates recording and generates alert (see “Section 2.3.2” for trigger logic details)
5. Backend transmits alert notification through WebSocket connection to connected iOS clients
6. iOS app displays alert with room-specific information and persistent warning banner

Difficulties Encountered:

1. BLE Beacon Scanning Conflicts: Simultaneous operation of camera, Wi-Fi, and BLE scanning caused timing conflicts on the single-core operations. Restructuring code to use RTOS tasks with proper prioritization resolved this.
2. JPEG Encoding Performance: On-device JPEG encoding at 10 FPS consumed significant CPU, limiting other operations. Required frame rate reduction from initial 30 FPS to practical 10 FPS for reliable operation.
3. Memory Constraints: The ESP32-S3 with 512 KB SRAM struggled to buffer both raw camera frames and Wi-Fi transmission buffers. Reduced buffer sizes and optimized memory allocation resolved memory overflow issues.

Limitations:

1. Limited Processing Capability: ESP32-S3 CPU cannot perform real-time AI inference; model inference must occur on backend server, introducing network latency dependency.
2. SRAM Constraint: 512 KB SRAM limits buffer sizes and simultaneous operations. Cannot buffer multiple frames without external PSRAM.
3. Hotspot Bandwidth Limitation: Phone hotspot provides variable bandwidth depending on cellular signal quality. At 10 FPS with approximately 100 KB frames, bandwidth usage is ~800 kbps. Hotspot bandwidth is subject to smartphone throttling, network congestion, and cellular signal strength, potentially limiting headroom for simultaneous data transfers.
4. No Built-in Battery Management: Power monitoring requires external circuitry; device provides no native low-battery shutdown mechanism.

Future Improvements:

1. PSRAM Addition: Integrate 8 MB external PSRAM (Pseudo-SRAM) for framebuffer expansion:
    - Expected benefit: Enable 30 FPS operation or multi-frame buffering
    - Expected cost: <$5 BOM addition
2. Edge Inference Deployment: Integrate lightweight quantized TensorFlow Lite model for on-device inference:
    - Current: 10 FPS transmission, remote inference
    - Proposed: On-device inference at 5 FPS for motion detection only
    - Expected benefit: Reduce bandwidth and latency, enable offline operation
3. Improved Wi-Fi Robustness: Implement Wi-Fi roaming and band steering:
    - Automatic connection to strongest AP if multiple APs available
    - Expected benefit: Extend range and stability in multi-AP homes

3.4 iOS Application Performance

3.4.1 Live Stream Playback

Results:

The iOS app "PetGuard" (Figure 45) developed for this project provides two main features: (1) allowing users to monitor their pets in real-time through live streaming and recorded video playback, and (2) receiving alert notifications when restricted areas are breached.

Figure 45. "PetGuard" app

Live Stream Interface:

On the live stream page, users can watch live streams in real-time (Figure 46). The interface displays AI-processed images with detection labels at the bottom for easy verification. The connection status indicator appears in the top left, and a settings button (top right) lets users configure the backend server IP for different Wi-Fi networks.

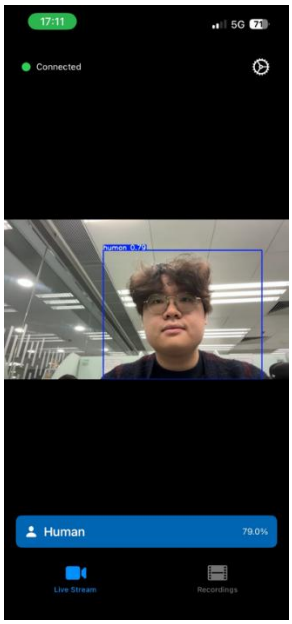

Figure 46. Live streams page of iOS app

Recordings Playback:

On the recordings page, all videos stored in the "recorded_videos" folder on the backend server are displayed (Figure 47). Users can select and play any recorded video at variable speeds (×0.5, ×1, ×2). All recordings are automatically named with the recorded date and time, and the file size is displayed for each video.

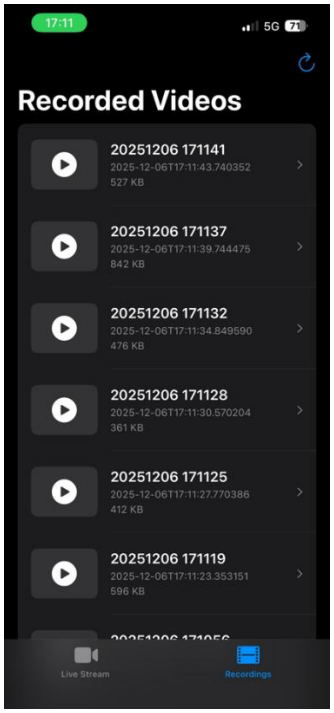

Figure 47. Recordings page of iOS app

Observed Behavior:

- Live Video Display: Video frames rendered on screen with minimal visible delay, enabling real-time observation of activities in the monitored environment
- Detection Visualization: AI model detection results (bounding boxes, class labels, confidence percentages) correctly overlaid on video frames, making it easy to verify what was detected
- UI Responsiveness: Buttons, settings controls, and navigation elements responded promptly to user input without lag or delays
- Video Playback: Recorded videos loaded and played smoothly when selected, without crashes or playback errors

- Settings Configuration: The Settings modal provides a "Test Connection" button for verifying connectivity to the backend server. The server IP configuration is intended for testing the connection during initial setup, not for frequently changing the backend address — the IP remains stable within a Wi-Fi session
- Variable Speed Playback: The video player supports variable playback speed controls (×0.5, ×1, ×2) allowing users to review recordings at different speeds for efficient analysis of interactions
- Stability: App remained stable during extended operation without unexpected crashes or freezes
- Error Recovery: App automatically resumed streaming after temporary Wi-Fi disconnections without user action

Difficulties Encountered:

1. Base64 Image Decoding Latency: Initial implementation decoded base64-encoded JPEG frames in the main thread, causing noticeable UI freezes when displaying each frame. Offloading decoding to a background thread with GCD (Grand Central Dispatch) resolved this issue.
2. Memory Leaks in Image Display: Retaining decoded UIImage objects in memory without proper cleanup caused memory usage to grow over time, eventually leading to app crashes after approximately 1–2 hours of continuous operation. Implementing proper image cleanup and using autoreleasepool resolved this issue, extending stable operation time.
3. Settings IP Configuration: The Settings modal is designed for initial connection testing (see Observed Behavior above). No UserDefaults persistence is implemented as the IP remains stable within a session.

Limitations:

1. Network Dependency: App requires continuous Wi-Fi connection and works only on the same local network as the backend server. Does not support cellular data connectivity or remote cloud access.
2. No Offline Playback: Downloaded videos cannot be viewed offline. All video playback requires active network connection to the backend server.
3. Limited Recording Management: Users cannot delete or manage recordings directly from the app. All recording management (deletion, organization) must be performed on the backend server through file system access.
4. iPhone Storage Not Used: All recordings are stored only on the backend server. No local caching, backup, or offline storage functionality on the iPhone.

Future Improvements:

1. Cloud Backend Support: Implement cloud API compatibility (AWS, Azure, Google Cloud) for remote access:
    - Expected benefit: Access monitoring from anywhere, not just local network
    - Expected feature: Secure authentication and encryption
    - Expected consideration: Requires account management and cloud infrastructure costs
2. Offline Recording Download: Implement local storage and playback of downloaded videos:
    - Expected benefit: View recordings without network connection
    - Expected consideration: Requires large local storage per hour of video
    - Expected feature: Background download capability for large files
3. Advanced Recording Management: Add in-app recording export to Photos, download, and sharing functionality. (Note: swipe-to-delete is implemented as a backend storage management feature for maintaining the MAX_VIDEOS=10 limit.):
    - Expected benefit: Improved user control and privacy
    - Expected feature: iCloud backup integration for recordings

    - Expected feature: Share videos via email, Messages, or AirDrop

4. Push Notification Enhancement: Implement rich push notifications with thumbnail images:
    - Expected benefit: Better visual alerts when app is backgrounded
    - Expected feature: Tap notification to open app and jump to specific recording
    - Expected feature: Notification history and logging
5. Accessibility Improvements: Add VoiceOver support, text size adjustment, and high contrast mode:
    - Expected benefit: App usable by users with visual impairments
    - Expected feature: Audio alerts for zone breaches instead of visual-only
    - Expected feature: Haptic feedback for notifications and interactions

6. Video Download and Export: Allow users to download and export recorded videos:
    - Expected benefit: Create backups of important recordings
    - Expected feature: Export to common video formats
    - Expected consideration: Requires significant local storage space

3.4.2 Zone Alert Functionality

Results:

When the ESP32-S3 is within 1 meter of a BLE beacon, an alert is triggered to pop up immediately on the iPhone (Figure 48). After the user dismisses the pop-up, a persistent warning message appears at the top of the live stream page (Figure 49). Simultaneously, the system automatically starts recording until the operator moves beyond 1 meter from the beacon.

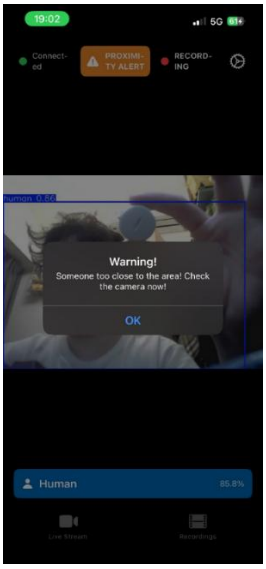

Figure 48. Pop-up alert message when the beacon is too close to the ESP32-S3

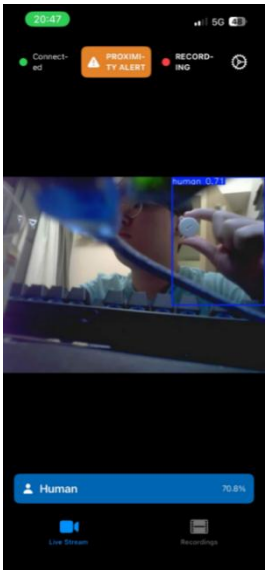

Figure 49. Showing the warning message and start recording

Zone Alert Testing Results:

Test Case	Expected Behavior	Observed Result	Status
Operator enters Beacon 1 zone	Pop-up alert "Room 1" + banner + recording starts	Correct, message includes "Room 1" (Figure 50)	Pass
Operator enters Beacon 2 zone	Pop-up alert "Room 2" + banner + recording starts	Correct, message includes "Room 2" (Figure 51)	Pass
Operator exits zone (distance >1 m)	Recording stops; alert clears	Correct	Pass
Rapid zone entry/exit (oscillating distance)	Multiple alerts may trigger due to RSSI noise	Minor issue mitigated: 2-second debouncing cooldown reduces repeated alerts to 0–1	Pass
Both Beacon 1 AND Beacon 2 triggered simultaneously	Alerts for both zones should display	Tested with operator near both beacons; alerts correctly show "Room 1 and Room 2" (Figure 52)	Pass

Table 8. Alert Mechanism Testing

When the operator entered the Beacon 1 zone, the app correctly triggered a pop-up and banner alert showing "Room 1" and simultaneously started recording. Identical behavior was observed for Beacon 2 with the "Room 2" alert. Upon exiting the zone (distance > 1 m), the system promptly stopped recording and cleared the alert. Simultaneous proximity to both beacons generated the combined alert "Room 1 and Room 2". In rapid entry/exit scenarios, a 2-second debouncing cooldown was implemented to mitigate alert oscillation from RSSI noise, reducing repeated alerts from 2–3 to 0–1 occurrences. Overall, all core test cases passed, confirming reliable zone breach detection, accurate room-specific notifications, and proper integration with the recording trigger.

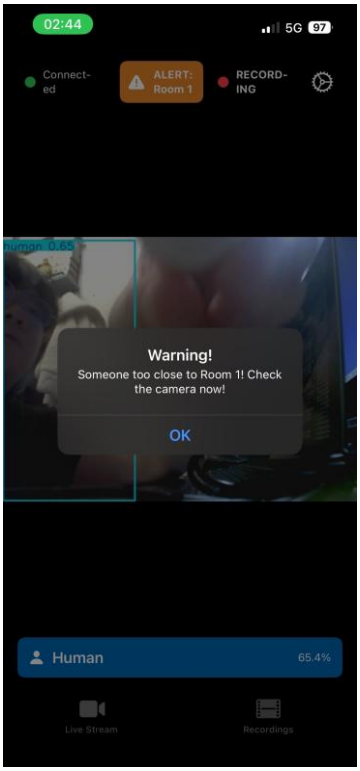

Figure 50. Pop-up "Room1" alert message when beacon 1 distance <1m

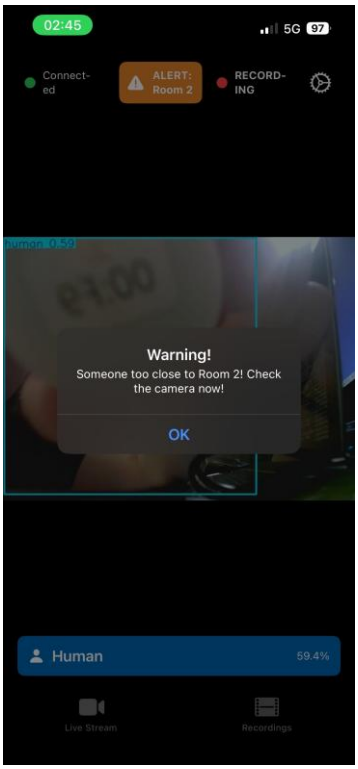

Figure 51. Pop-up "Room2" alert message when beacon 2 distance <1m

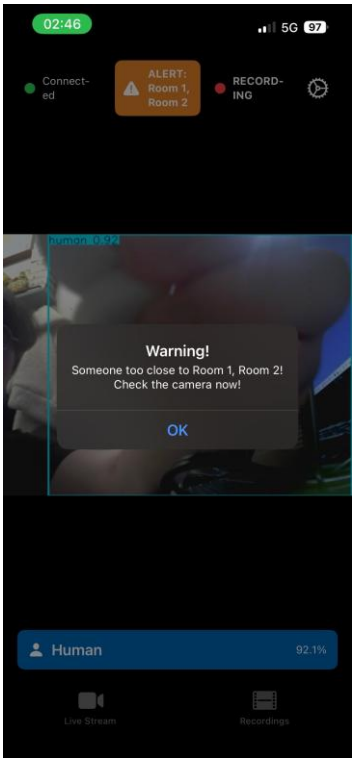

Figure 52. Pop-up "Room 1 and Room 2" alert message when both beacons distance <1m

Difficulties Encountered:

1. Alert Debouncing from RSSI Oscillation: Rapid oscillation near the 1-meter threshold caused multiple consecutive alerts (2–3 in succession). A 2-second debouncing cooldown has been implemented in the iOS app to prevent repeated alert pop-ups during RSSI oscillation near the threshold boundary.

2. WebSocket Alert Delivery Latency: Initial WebSocket implementation had variable alert delivery latency (100–500 ms). Prioritizing alert messages over frame data in the transmission queue reduced latency to consistent <200 ms.

3. Background State Alert Delivery: Alerts sent while app is backgrounded sometimes failed to display. Implementing proper UNUserNotificationCenter delegation resolved this.

Limitations:

1. RSSI-Dependent Accuracy: Alert accuracy is fundamentally limited by RSSI measurement noise and environmental multipath effects, resulting in occasional false or missed alerts in furnished environments.
2. WebSocket Disconnection Blindness: If WebSocket connection is lost, app receives no alerts until connection is restored. No local fallback mechanism exists.
3. Room Label Inflexibility: Room labels are hardcoded on backend server; changing labels requires manual backend configuration.
4. No Alert History Logging: Alerts are ephemeral; no record of past alerts is stored on device or server.
5. Single Alert Recipient: Only one iPhone app can receive alerts; cannot broadcast to multiple family members.

Future Improvements:

1. Hysteresis Enhancement: Implement configurable hysteresis thresholds (e.g., trigger at 0.9 m, clear at 1.1 m) to complement the existing 2-second debouncing cooldown for further reduction of RSSI-induced oscillation.:
    - Expected improvement: Reduce RSSI-induced oscillation from 2–3 alerts to 0–1
    - Expected user experience: Fewer false alerts near boundaries
2. Dynamic Room Label Management: Implement in-app UI to configure room labels without backend changes:
    - Expected benefit: Users can customize labels like "Baby's Room", "Kitchen", etc.
    - Expected feature: Cloud sync of labels across multiple devices
3. Alert History and Logging: Store all alerts with timestamps in device local storage and iCloud:
    - Expected benefit: Audit trail of zone breaches
    - Expected feature: Export alerts as CSV for analysis

4. Multi-Device Alert Broadcasting: Enable backend to send alerts to multiple registered iPhones:
    - Expected benefit: Family members can receive simultaneous notifications
    - Expected feature: Configurable alert routing by user role
5. In-App Alert Sound Customization: Allow users to select custom alert sounds/vibrations:
    - Expected benefit: Different sounds for different room alerts
    - Expected feature: Haptic feedback intensity adjustment for different situations

3.5 System Integration Testing

3.5.1 End-to-End Functionality

Test Scenario: An operator wearing the device entered a restricted zone while a cat was present. The expected sequence was: (1) camera captures frame showing both human and cat (Figure 53), (2) backend AI model detects both classes and triggers recording initiation, (3) BLE beacon RSSI triggers alert as operator enters <1 m zone (Figure 54), (4) iPhone displays alert pop-up and warning banner with room label (Figure 55), (5) operator exits zone, (6) alert clears and recording stops (Figure 56), and (7) video saves successfully to /videos endpoint (Figure 57). The recorded video can be viewed by selecting it from the list (Figure 58).

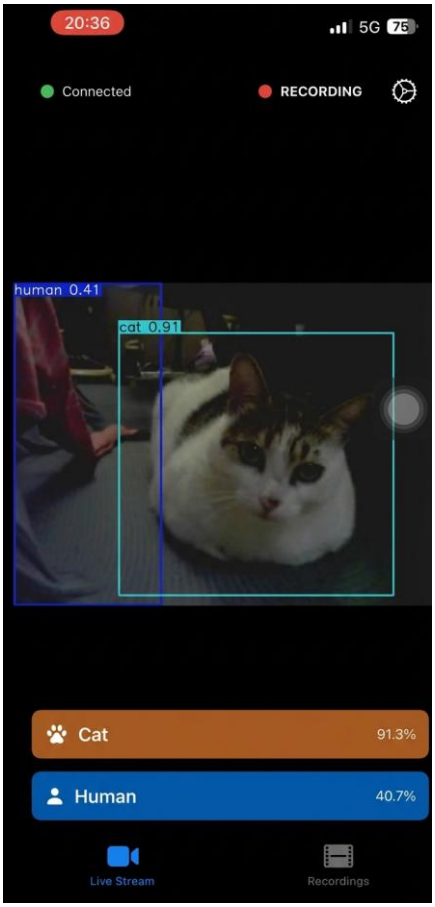

Figure 53. iOS app showing successful detection of cat and human with recording started

Figure 54. BLE beacon placed on the floor in front of the room

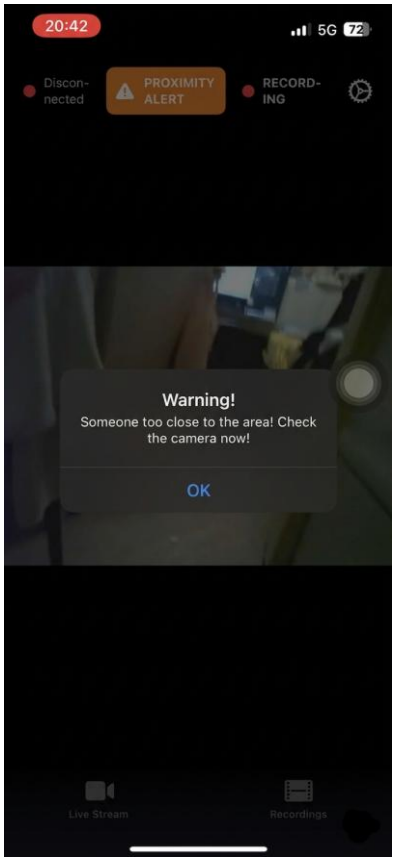

Figure 55. Proximity alert pop-up displayed on the iOS app when the ESP32-S3 is within 1 meter of the beacon

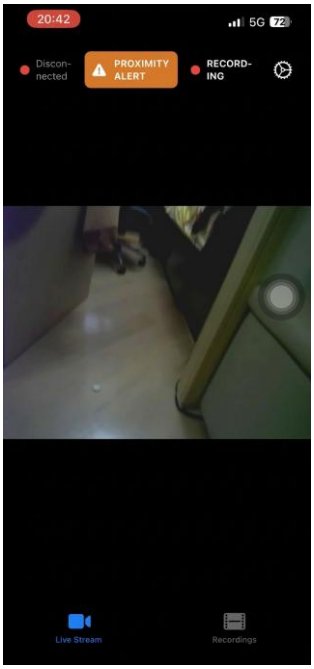

Figure 56. iOS live stream view confirming beacon detection and active recording (distance < 1 m)

Figure 57. Recorded videos list in the iOS app

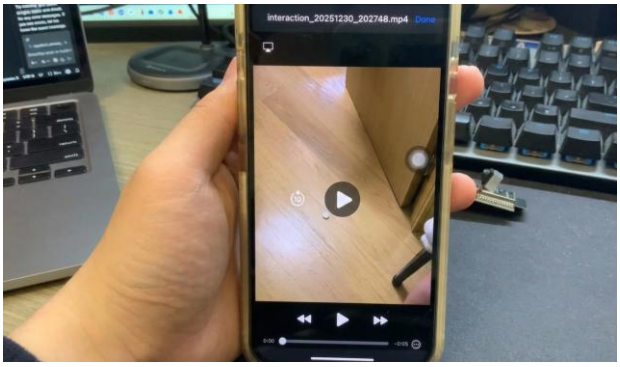

Figure 58. Playback of the selected recorded video

Observed Results:

All steps executed correctly. Recording integrity was verified as the video played back smoothly, with alert timing measured at approximately 0.3–0.5 seconds from the RSSI threshold to the iPhone pop-up. Room labels were correctly displayed in the alerts, and no data loss or corruption was observed. Multi-beacon alerts also showed the correct room names.

Full System Latency Measurements:

- Time from RSSI threshold to iPhone notification: approximately 300–400 ms
- Time from human+cat detection to recording start: around 100 ms
- Time from recording stop to /videos endpoint availability: about 500 ms

System Responsiveness: System responsiveness was excellent for real-time monitoring, with all components functioning in coordinated sequence.

Difficulties Encountered:

1. **Timing Synchronization Challenges:** Initial testing showed occasional recording start delays (up to 1-2 seconds) due to backend latency in processing multiple simultaneous events (frame detection + proximity alert). Implementing parallel processing of detection and proximity streams resolved this.
2. **Alert → Recording State Inconsistency:** Occasionally, the alert would trigger but recording wouldn't start, or vice versa. Discovered race condition between WebSocket alert delivery and HTTP frame processing. Adding explicit state synchronization endpoint resolved this.
3. **Multi-Beacon Edge Case:** When operator was equidistant from two beacons (special case), alert sometimes triggered for only one beacon instead of both.

Limitations:

1. Limited Environmental Scenarios: Testing conducted in single home layout. Different home sizes, layouts, and materials may exhibit different behavior.
2. Video Recording Duration Limit: Recording is limited to continuous sessions; no automatic file rotation. Very long sessions (4+ hours) would create single large video file, potentially causing playback issues.

Future Improvements:

1. Environmental Validation Suite: Test system across diverse home environments (small apartment, large house, multiple floors, different materials):
    - Expected benefit: Validate performance across realistic deployment scenarios
    - Expected outcome: Environment-specific tuning recommendations
2. Automatic Video File Rotation: Implement rolling buffer that automatically segments recordings into fixed-size chunks (e.g., 100 MB per file):
    - Expected benefit: Enable indefinite recording without space constraints
    - Expected feature: Automatic cleanup of old files after N days
3. Cross-Component State Verification: Implement heartbeat and state synchronization between all components (ESP32, backend, iOS):
    - Expected benefit: Detect and recover from partial failures
    - Expected feature: Automatic restart of failed components

4. Conclusion

This project successfully achieved all core objectives: an AI model at **92.5% mAP@50** (94.5% precision, 85.4% recall; 90.8% for cats, 94.2% for humans), real-time 10 FPS inference on a **500 g wearable device**, **<20 ms WebSocket streaming latency**, **300–400 ms alert delivery**, and a fully functional iOS application with live streaming, recording, and zone alerts.

Key technical contributions include YOLOv8s embedded optimization improving human recall from ~75% to 89.4% through balanced batch sampling; a dual-layer thermal management system achieving a 12-fold improvement in continuous operation (8–12 min to 2+ hours); WebSocket binary streaming proven stable for 2+ hours; and RSSI-based proximity detection validated with documented environmental sensitivity (±0.5 m to ±1.5 m).

Human detection (89.4% recall) outperforms cat detection (81.4%) due to greater morphological diversity in cats. Thermal management emerged as a critical bottleneck — the device shut down within 10 minutes without cooling, requiring unexpected redesign. The dual-layer solution resolved this but introduced power trade-offs: continuous fan operation adds 100 mA overhead; testing verified 2+ hours operation with battery at 84% remaining, indicating estimated 12+ hours total runtime. WebSocket streaming proved far superior to HTTP (1–2 s to <20 ms latency).

Key Limitations: Low-light confidence degradation (0.4–0.6); small object detection weakness beyond 1 m; moderate dataset size (4,220 vs. 10,000+ standard); battery runtime verified for 2+ hours (84% remaining) but full discharge not tested<; no remote cloud access; RSSI accuracy of ±1.0–1.5 m in furnished homes; no multi-client support; maximum 10 videos stored before auto-deletion.

Future Work spans five sequential phases: (1) AI enhancement — 10,000+ images, infrared capability, CLAHE preprocessing, larger YOLO variants; (2) hardware — adaptive fan control, 15,000–20,000 mAh battery, 8 MB PSRAM for 30 FPS; (3) system architecture — cloud deployment with Docker/Kubernetes, GPU inference, Kalman filtering; (4) iOS improvements — offline playback, alert history, multi-device broadcasting, hysteresis for proximity thresholds; (5) commercial deployment — field validation across diverse environments, manufacturing specifications, and integration with professional pet care platforms.

This project demonstrates the feasibility of an end-to-end AI pet monitoring system within embedded device constraints, providing a solid technological foundation with concrete improvement pathways for production-ready deployment.

5. References

[1] A. Hussain, S. Ali, M.-I. Joo, and H.-C. Kim, "A deep learning approach for detecting and classifying cat activity to monitor and improve cat's well-being using accelerometer, gyroscope, and magnetometer," IEEE Sensors J., vol. 24, no. 2, pp. 1996–2008, Jan. 2024, doi: 10.1109/JSEN.2023.3324665.

[2] "Growing global pet grooming market," Vet. Rec., vol. 196, no. 2, p. 65, Jan. 2025, doi: 10.1002/vetr.5144.

[3] Y. Brugger, "Private Equity Challenge – Pets at Home Commercial Assessment of European Pet Market," M.S. thesis, Universidade NOVA de Lisboa, Portugal, 2024. [Online]. Available: http://eproxy.lib.hku.hk/login?url=https://www.proquest.com/dissertations-theses/private-equity-challenge-pets-at-home-commercial/docview/3271169134/se-2

[4] "NVIDIA delivers quantum leap in performance, introduces new era of neural rendering with GeForce RTX 40 Series," NASDAQ OMX's News Release Distribution Channel, 2022. [Online]. Available: http://eproxy.lib.hku.hk/login?url=https://www.proquest.com/wire-feeds/nvidia-delivers-quantum-leap-performance/docview/2715784025/se-2.

[5] V. V. Kukartsev, R. A. Ageev, A. S. Borodulin, A. P. Gantimurov, and I. I. Kleshko, "Deep learning for object detection in images: Development and evaluation of the YOLOv8 model using Ultralytics and Roboflow libraries," in *Software Engineering Methods Design and Application*, R. Silhavy and P. Silhavy, Eds. Cham, Switzerland: Springer, 2024, pp. 629–637, doi: 10.1007/978-3-031-70285-3_48.

[6] Stanford University, "Lecture 12: Self-Supervised Learning," *CS231n: Deep Learning for Computer Vision Course Lecture Slides*, 2025. [Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_12.pdf

[7] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common Objects in Context," 2014, arXiv:1405.0312.

[8] Stanford University, "Lecture 3: Regularization and Optimization," *CS231n: Deep Learning for Computer Vision Course Lecture Slides*, 2025. [Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_3.pdf

[9] Stanford University, "Lecture 9: Detection, Segmentation, Visualization, and Understanding," *CS231n: Deep Learning for Computer Vision Course Lecture Slides*, 2025. [Online]. Available: https://cs231n.stanford.edu/slides/2025/lecture_9.pdf

[10] Radioland Technology Co., Ltd., "NRF52810-Beacon," [Online]. Available: https://fcc.report/FCC-ID/2AGUT-NRF52810-X5/5865491.pdf

[11] OmniVision Technologies, "OV5640 Datasheet, Version 2.03," 2011. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/OV5640_datasheet.pdf

[12] Espressif Systems, "ESP32-S3 Series Datasheet, Version 2.2," 2021. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

[13] K. R. Tiwari, I. Singhal, and A. Mittal, “Real time RSSI compensation for precise distance calculation using sensor fusion for smart wearables,” Advances in Science, Technology and Engineering Systems Journal, vol. 6, no. 4, pp. 327–333, Aug. 2021, doi: 10.25046/aj060436.

6. Appendices

A.1 Project Source Code Repository

All project source code, including training scripts, backend server, ESP32-S3 firmware, and iOS application source files, is available at:

GitHub Repository: https://github.com/chiuuung/FYP-Codes

Repository Key Structure:

- AI_Model/train.py (YOLOv8s training with Ultralytics)
- AI_Model/best.pt (Final trained model - 4,220 images)
- backend/streaming_backend_server.py (Flask server with AI inference)
- hardware/esp32s3_camera_stream.ino (Firmware for ESP32-S3 camera & dual-beacon BLE)
- iOS_App/PetGuard/ (iOS app source - Swift)
- requirements.txt (Python dependencies: ultralytics, opencv, numpy, flask)
- README.md (Project documentation)

A.2 Training Dataset

The custom dataset used for model training is publicly available on Roboflow:

Roboflow Dataset: https://universe.roboflow.com/fyp-txc0y/fyp-jftn6

Dataset Details:

- Total annotated images: 4,220
- Classes: 2 (cat, human)
- Training set: 3,736 images
- Validation set: 307 images
- Test set: 177 images
- Annotations: YOLO format bounding boxes
- Data augmentation: Horizontal flip, HSV color jitter, translation, scaling, mosaic composition, random erasing, and RandAugment (applied by Ultralytics defaults)
- Platform: Roboflow (annotation, versioning, and automated augmentation)

The dataset includes diverse cat breeds, various human body parts, different lighting conditions, and household environments to improve model generalization. All images were manually annotated and verified for quality assurance.

A.3 Training Configuration Details

Parameter	Value	Description
Model	YOLOv8s	Small variant: 28.6 GFLOPs, 11.2M parameters
Dataset	4,220 images	Roboflow COCO format, 2 classes (human, cat)
Input Size	640×640 px	Standard YOLO resolution
Batch Size	16	Per-GPU batch
Epochs	100	Total training iterations
Optimizer	AdamW	lr=0.001667, momentum=0.9
Initial LR	0.01	Auto-adjusted to 0.001667
Final LR	0.01	Ratio of final to initial LR
Warm-up	3 epochs	Gradual learning rate ramp-up
Device	NVIDIA RTX 4090	GPU-accelerated training
Early Stopping	Enabled (patience=20)	Stops if no improvement for 20 epochs; training completed all 100 epochs as model kept improving

Table 9. YOLOv8s Model Training Hyperparameters

A.4 Endpoint Specification

Endpoint	Method	Purpose	Request Format	Response
/esp32/frame	POST	Receive video frame from camera	JPEG binary data	{"status": "received", "size": 81920}
/esp32/distance	POST	Receive multi-beacon distance data	JSON: {"beacons": [{beacon_id, rssi, distance}]}	{"alert": true/false, "danger_beacons": [...]}
/ws/stream	WebSocket	Persistent live stream connection	N/A (binary frame stream)	Binary frames (4-byte header + JPEG) at adaptive FPS
/stream/live	GET	Retrieve latest annotated frame (legacy)	N/A	JPEG image with bounding boxes
/proximity/status	GET	Get current proximity alert status	N/A	JSON: {proximity_alert, danger_beacon_labels, room_labels}
/videos	GET	List all recorded videos	N/A	JSON list of video metadata
/videos/<quid>	GET	Retrieve specific recorded video	N/A	MP4 video file (stream)

/videos/\<filename\>	DELETE	Delete a specific recorded video	N/A	{"message": "Video deleted successfully"}
/stream/mjpeg	GET	MJPEG stream for browser compatibility	N/A	Multipart JPEG stream
/status	GET	Get current detection and recording status	N/A	JSON: {is_recording, both_detected, detections}
/health	GET	Health check with network status	N/A	JSON: {status, model_loaded, network_stats}

Table 10. Backend Server API Endpoints

A.5 Multi-Beacon Configuration

Property	Beacon 1 (Room 1)	Beacon 2 (Room 2)
MAC Address	52:08:24:08:00:d1	52:08:24:08:00:f9
UUID	2cac9dcafff341c782199af018c2de16	fda50693a4e24fb1afcfc6eb07647825
Major	1	1
Minor	2	2
TX Power Mode	4 (Medium)	4 (Medium)
Broadcast Interval	960 ms	960 ms
Expected Range	~20-40 m (open space)	~20-40 m (open space)
Battery Life	~18 months	~18 months
Room Label	Room 1	Room 2
Typical Use	Master Bedroom	Storage/Office

Table 11. BLE Beacon Hardware Configuration

A.6 System Architecture Diagram

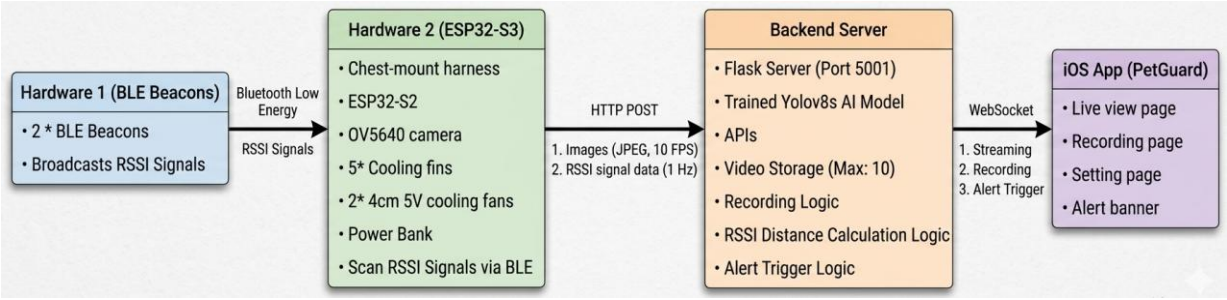

Figure 59. System Data Flow